CHAPTER 1: INTRODUCTION

1.1 Background

In recent years, the healthcare industry has seen a significant digital transformation. With the rise of smartphones and internet access, mobile applications have become essential tools for accessing medical services. One such advancement is the development of online pharmacy Website like PharmEasy, which allow users to purchase medicines, book diagnostic tests, and consult doctors online. The need for such platforms became especially critical during the COVID-19 pandemic when access to physical pharmacies and hospitals was restricted. This project aims to replicate and simplify the core functionalities of such an application, making healthcare more accessible, affordable, and efficient for users.

1.2 Objectives

1. To develop an intuitive and secure medical application for medicine delivery.
2. To provide users with easy access to a wide range of genuine medicines and healthcare products.
3. To simplify the process of uploading prescriptions and verifying them digitally.
4. To offer lab test booking and doctor consultation options through the app.
5. To ensure timely delivery and tracking of medicine orders.
6. To create a robust backend system for order, inventory, and user data management.

1. To develop an intuitive and secure medical application for medicine delivery

This objective focuses on building a user-friendly app that is easy to navigate for all age groups. Security features will be integrated to protect user data and transactions. The goal is to ensure a smooth and safe medicine ordering experience.

2. To provide users with easy access to a wide range of genuine medicines and healthcare products

The app will offer a vast catalog of verified medicines, health supplements, and personal care items.

Users can search, filter, and find products based on categories, brands, or ailments. This ensures convenience and trust in product authenticity.

3. To simplify the process of uploading prescriptions and verifying them digitally

Users will be able to upload doctor prescriptions directly through the app.
These prescriptions will be reviewed and validated by licensed pharmacists. This streamlines the ordering process and ensures compliance with medical regulations.

1. To offer lab test booking and doctor consultation options through the app

The app will allow users to schedule diagnostic tests with home sample collection. Additionally, virtual doctor consultations can be integrated for better treatment support. This provides a complete digital healthcare solution in one place.

2. To ensure timely delivery and tracking of medicine orders

The system will include real-time order tracking to keep users updated on delivery status.
Logistics will be managed efficiently to ensure quick and safe delivery.
Notifications and updates will enhance the user experience throughout the process.

3. To create a robust backend system for order, inventory, and user data management

A powerful backend will handle orders, product inventory, user accounts, and transactions.
Admins and pharmacists will manage stock levels, prescriptions, and delivery through this system.
It will ensure smooth operations and data integrity across the platform importance of medicines in a medical. medicines are the core component of any healthcare service. In a medical app, the inclusion of a medicine module brings immense value by

1. Providing immediate access to prescribed and over-the-counter (OTC) drugs.
2. Helping patients manage chronic illnesses by ensuring timely refills.
3. Offering drug information, dosage, side effects, and usage instructions.
4. Supporting elderly or remote patients who may struggle to visit a physical pharmacy.

5. Enabling discounted pricing, monthly subscriptions, and bulk orders for affordability.
6. Acting as a central database for healthcare history, prescription logs, and diagnostics.
7. A medical app without a medicine component remains incomplete, as medicine is often the first line of treatment after diagnosis.

1.3 Definition

A Medical App with Pharmacy Features is a digital platform designed to help users search, purchase, and manage medicines online, along with features such as prescription upload, dosage reminders, and order tracking. These Website bridge the gap between patients and pharmacies by offering a seamless, remote, and secure method to access essential healthcare products and services.

1.4 Purpose, Scope, and Applicability

Purpose:

I. To digitize and streamline the process of buying medicines.
II. To ensure patient convenience through home delivery and prescription management. III. To improve healthcare accessibility through a single unified platform.

The primary purpose is to replace traditional pharmacy visits with a smooth digital experience.
Users can search, order, and receive medicines through a few simple steps on the app.
This reduces waiting time, minimizes errors, and improves the overall efficiency of the process.

The app aims to make healthcare services accessible from the comfort of one's home. Users can upload prescriptions, track orders, and get medicines delivered safely. This is especially beneficial for elderly patients, busy professionals, and those with mobility issues.

.Scope:

I. Medicine ordering (prescription and non-prescription)

II. Upload and digital validation of prescriptions
III. Inventory management by pharmacists
IV. User authentication and secure payments
V. Diagnostic test booking and teleconsultation (if extended)

I. Diagnostic test booking and teleconsultation (if extended)

The app may offer lab test booking with options for home sample collection.
Additionally, it can support virtual doctor consultations through chat or video. This expands the app's utility from medicine delivery to a complete healthcare platform.

Applicability:

I.Applicable for individuals with chronic diseases needing regular medication. II. Useful in remote areas with limited access to physical medical stores.
III. Convenient for working individuals, senior citizens, and caregivers.
IV. Can be used by hospitals, clinics, and diagnostic centers for bulk ordering and logistics.

Patients with chronic conditions like diabetes, hypertension, or asthma require continuous medication.
The app enables easy ordering and timely refills without visiting a pharmacy. This

1.5 Project Objectives

The main objective of this project was to enable customers to choose and buy the desired medications with the help of kiosk software. We had to develop a real-time platform and integrate it with third-party solutions like Chrome and WebRTC. In addition, we were asked to add a video call feature to enable customer-pharmacist consultations and enhance customer experience.

1. To design and develop a user-friendly interface for a pharmacy app.
2. To allow users to register, log in, and browse a categorized list of medicines.

3. To implement a system for uploading prescriptions and validating them.
4. To integrate a cart, checkout, and payment module.
5. To build an admin panel for managing inventory, orders, and user data.
6. To test the application for functionality, performance, and user experience.
7. To document the project with system architecture, technology stack, workflows, and future enhancements.

CHAPTER 2: SURVEY OF TECHNOLOGIES

Choosing the right technology is an important part of the development process. The original project was loaded in **Visual Studio 2010** for design and coding, and it used **SQL Server 2008** for database management.

For the modern Online Medical Store web application, a different, more contemporary, and efficient technology stack has been selected. This chapter demonstrates the understanding and justification of the technologies chosen for the project, which follow a **MERN stack** (MongoDB, Express, React, Node.js) architecture.

2.1 Introduction

Developing the Online Medical Store requires a careful selection of robust, widely used, and efficient technologies. The aim is to build a platform that is highly responsive, maintainable, and uses a single language across the entire application stack (JavaScript).

For this project, the chosen technologies are:

- **React JS** — Frontend Development
- **Node.js** — Backend Runtime Environment
- **Express.js** — Backend Web Application Framework
- **MongoDB** — Non-Relational (NoSQL) Database Management

The entire project development is carried out using **Visual Studio Code** (IDE).

2.2 Frontend Technology: React JS

React JS is a popular, open-source JavaScript library for building user interfaces, developed and maintained by Facebook. It is used to develop a **Single Page Application (SPA)** for the Online Medical Store, ensuring a fast and seamless user experience.

Features and Examples

- **Component-Based Architecture:** The user interface is divided into reusable, isolated components (e.g., product cards, navigation bar, shopping cart), which makes the code modular and easier to manage.
- **Virtual DOM:** React maintains a lightweight, in-memory representation of the actual DOM (Document Object Model). When data changes, React calculates the most efficient way to update the browser's DOM, ensuring **faster rendering** and performance.
- **JSX:** A syntax extension that allows writing HTML-like code within JavaScript, facilitating the overall process of writing components.
- **SEO Friendly:** While SPAs can sometimes be challenging for search engines, React's ability to support server-side rendering makes it highly compatible with search engine optimization.

Advantages of using React JS

1. It facilitates the overall process of writing components.
2. It boosts productivity and facilitates further maintenance.
3. It ensures faster rendering.
4. It is SEO friendly.
5. It comes with a helpful developer toolset for debugging and development.

Styling and UI Libraries

For STYLING, **Bootstrap** and pure **CSS** are used.

Bootstrap is a Free and Open-Source CSS Framework directed at responsive, mobile-first front-end web development. It contains CSS- and (optionally) JavaScript-based design templates for typography, forms, buttons, navigation, and other interface components, ensuring a consistent and responsive layout across all devices.

2.3 Backend Runtime: Node.js

Node.js is a server-side platform built on **Google Chrome's JavaScript Engine (V8 Engine)**. It allows developers to run JavaScript code outside of a web browser, which is crucial for the backend of a web application.

Features and Examples

- **Asynchronous and Event-Driven:** Node.js uses a non-blocking I/O model, making it lightweight and efficient for handling concurrent connections, which is essential for a hightraffic e-commerce store.
- **Single-Language Stack:** By using Node.js, the project utilizes **JavaScript** for both the frontend (React) and the backend, simplifying development and making the entire stack cohesive.
- **V8 Engine Performance:** The V8 engine compiles JavaScript into native machine code, providing excellent execution speed.

2.4 Backend Framework: Express.js

Express.js, or simply **Express**, is a minimal and flexible Node.js web application framework, released as free and open-source software under the MIT License. It simplifies the process of building robust APIs and web applications with Node.js.

Features and Examples

- **Routing:** Handles the mapping of HTTP requests (GET, POST, PUT, DELETE) to specific functions (e.g., fetching a product list, processing an order).
- **Middleware:** Provides a mechanism for running functions before a final request handler, useful for tasks like user authentication, logging, and data validation.
- **API Development:** It is the standard framework for building **RESTful APIs** (Application Programming Interfaces) that allow the React frontend to communicate with the MongoDB database.

2.5 Database Technology: MongoDB

For the database and server, a **Non-Relational (NoSQL) database** is used.

MongoDB is an open-source NoSQL database management program. NoSQL is used as an alternative to traditional relational databases.

Features and Examples

- **Document-Oriented:** MongoDB stores data in flexible, JSON-like **BSON** documents. This makes it a great fit for the changing product, user, and order structures of an ecommerce application.
- **Scalability:** NoSQL databases like MongoDB are quite useful for working with large sets of distributed data and offer high availability and horizontal scaling.
- **Flexible Schema:** Unlike SQL, MongoDB does not enforce a rigid schema, allowing fields to vary from document to document, which speeds up development and iteration.
- **Seamless Integration:** It integrates naturally with the JavaScript/Node.js backend, making it a perfect component of the MERN stack.

2.6 Technology Comparison & Justification

Component	**Technology Chosen**	**Alternatives Considered**	**Justification**
Frontend	**React JS**	Angular, VueJS	Component-based, Virtual DOM for fast rendering, large ecosystem, and community support.
Backend Runtime	**Node.js**	Python, Java, PHP	Single-language stack (JavaScript) for full-stack development, asynchronous and non-blocking I/O for high performance.
Backend Framework	**Express.js**	Koa, NestJS	Minimalist, flexible, standard framework for Node.js API development.

Database	**MongoDB**	MySQL, PostgreSQL	Non-relational (NoSQL) for flexible data models, high scalability, and native integration with the Node.js environment.
Styling	**Bootstrap & CSS**	Tailwind CSS, Material UI	Provides a fast, responsive, and consistent design framework.
IDE	**Visual Studio Code**	Visual Studio, Sublime Text	Lightweight, powerful features, excellent JavaScript/React/Node.js support.

The chosen **MERN stack** (MongoDB, Express, React, Node.js) provides a robust, modern, and high-performance foundation for the Online Medical Store. The use of a single language—**JavaScript**—across the entire stack simplifies development, maintenance, and team collaboration, making it ideal for a student project while delivering a scalable e-commerce solution.

2.7 Conclusion

After evaluating technology options, **React JS** for the frontend, **Node.js** with **Express.js** for the backend API, and **MongoDB** for the database were selected. This stack ensures:

- **A secure, fast, and interactive user interface.**
- **High-performance, scalable backend services.**
- **Simplified development and maintenance using a unified JavaScript ecosystem.**

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

- Traditional medicine buying is inconvenient, time-consuming, and lacks transparency.
- Issues include geographical limits, long wait times, manual prescription handling, and lack of product info.
- The solution: a centralized, reliable online platform for medicine orders with secure prescription verification and fast delivery.
- Sub-problems: Prescription authenticity, inventory management, order tracking, and userfriendly interface.

3.2 Requirements Specification

Functional Requirements:

- User Management: Customer registration/login; Admin/Pharmacist dashboard.
- Product Catalog: Categorized display, detailed info, search/filter.
- Prescription Management: Upload prescriptions, pharmacist verification, restrict purchase until verified.
- Shopping Cart & Checkout: Manage cart, multiple payment options.
- Order Processing: Unique order IDs, real-time status updates, admin order management.

Non-Functional Requirements:

- Usability: Responsive, intuitive UI.
- Performance: Fast loading/search, handle peak loads.
- Scalability: Support growth via MERN stack.
- Security: Encrypt credentials and prescriptions, secure payments.
- Reliability: Consistent uptime and inventory accuracy.

3.3 Planning and Scheduling

- Phase 1: Requirement analysis & wireframes.
- Phase 2: System design (MERN stack architecture, database schema, API design).
- Phase 3: Backend development (Node.js/Express).
- Phase 4: Frontend development (React).
- Phase 5: Testing (unit, integration, UAT).
- Phase 6: Deployment & documentation.

3.4 Software and Hardware Requirements

Hardware:

- Processor: Intel i3 or equivalent
- RAM: 4GB minimum (8GB recommended)
- Storage: 256GB SSD
- Internet: Stable broadband **Software:**
- OS: Windows/Linux/macOS (latest stable)
- Backend: Node.js (LTS)
- Database: MongoDB (latest)
- Frontend: ReactJS (latest)
- Tools: VS Code, Git/GitHub, modern browsers

3.5 Preliminary Product Description

- Secure SPA for browsing and purchasing health products.
- Role-based access (Customer, Pharmacist/Admin).
- Prescription upload & verification system.
- Order tracking and history.
- Responsive design with Bootstrap for mobile accessibility.

Gantt Chart:

The Gantt chart illustrates the overall timeline and progress plan for the Online Medical Store (Pharmacy Web Application) project. It outlines each development phase — from requirement analysis to deployment — along with their respective durations and dependencies to ensure systematic project execution.

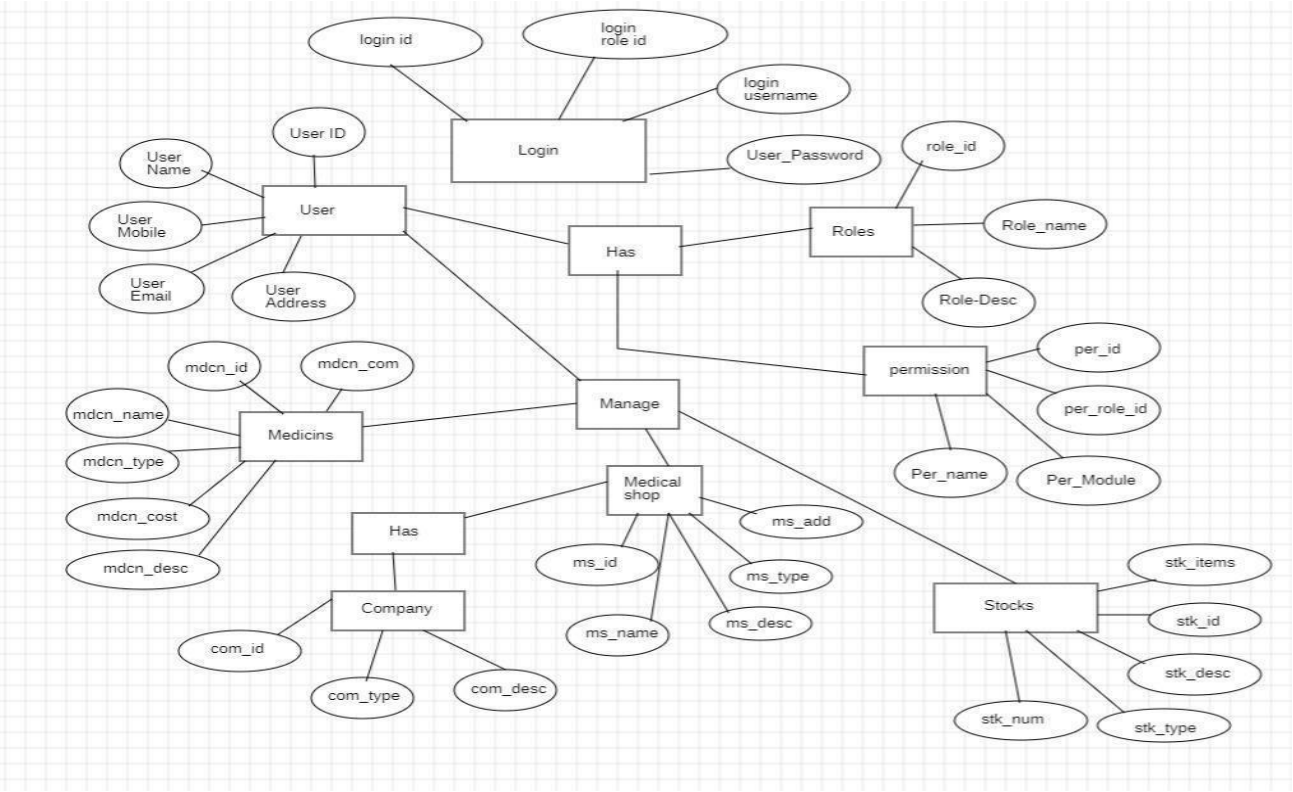
Date	21-06-2025	10-07-2025	24-08-2025	01-09-2025	15-09-2025	05-10-2025	17-10-2025
Project Scope Definition							
Proposal Approval Request							
Review of Pre-existing Methods							
Project Roadmap Creation							
Analysis							
Documentation							
Final Project Handover							

3.6 Conceptual Models

- **User:** Customers and Admin/Pharmacists (with roles and contact info).
- **Product:** Items with details like name, category, price, stock, prescription requirement.
- **Prescription:** Uploaded document with verification status linked to user.
- **Order:** Contains user, products, total amount, delivery address, order status.
- Relationships: Users place orders; orders contain products; prescriptions verified by pharmacist; admin manages inventory and order status.

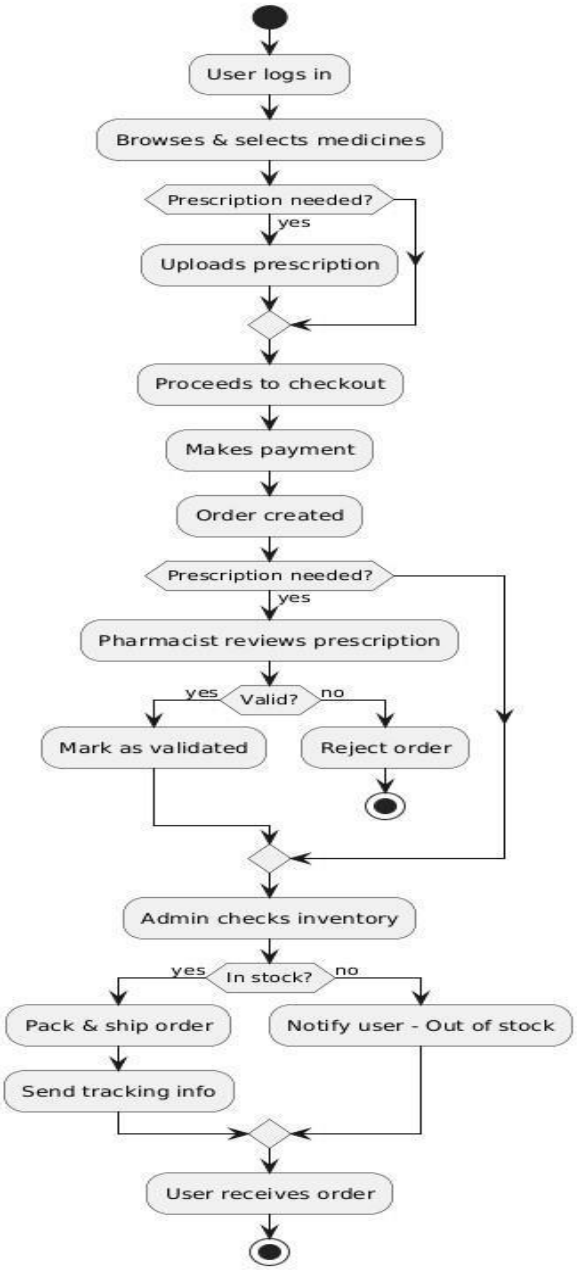
3.6.1 Entity Relationship (ER) Diagram:

This is the **Entity-Relationship Diagram (ERD)** for the Online Medical Store database. It shows how the main data pieces—**Users**, **Products**, **Orders**, and **Prescriptions**—connect. The diagram uses lines (relationships) to define rules, like how one **User** can place **many Orders**, and how each **Order** can contain **many Products** via the order_items table.



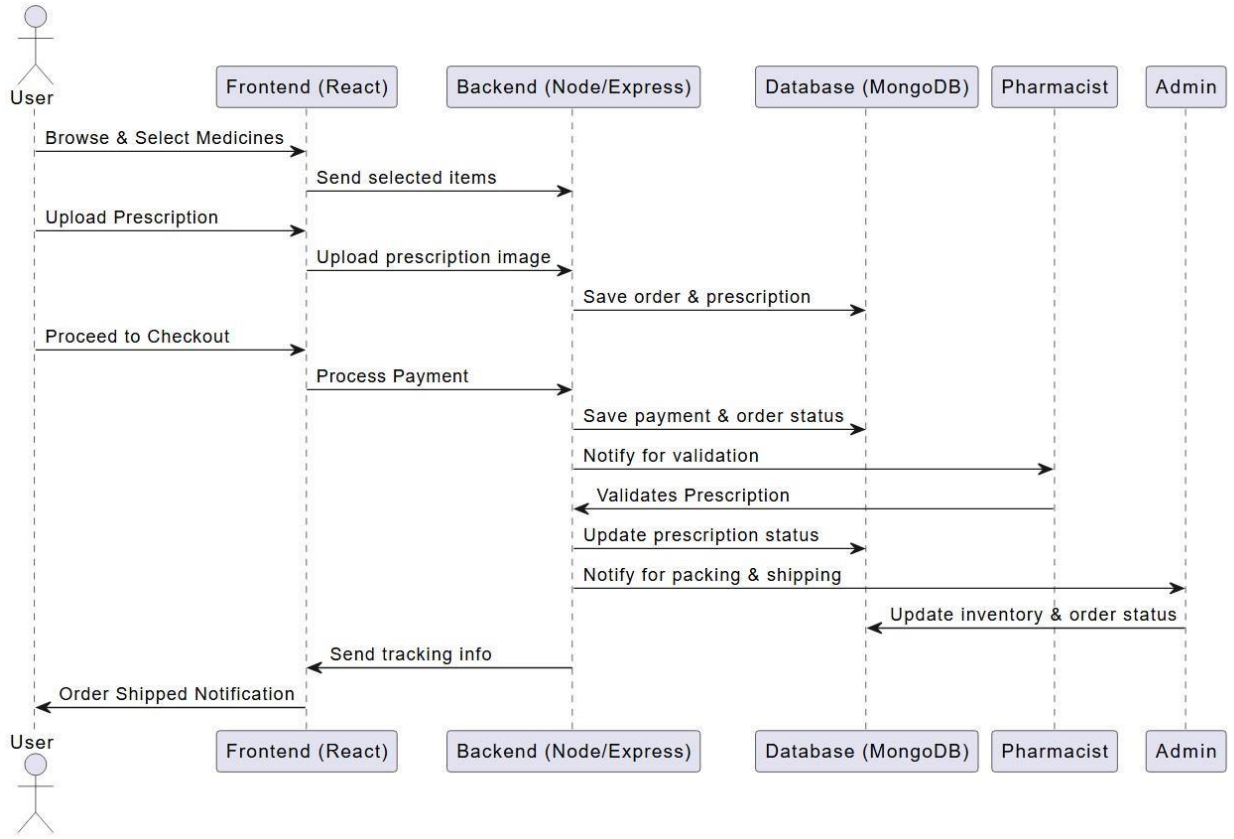
3.6.2 Activity Diagram:

This activity diagram illustrates the simplified workflow of an online pharmacy application. It covers key steps such as medicine selection, prescription upload, order placement, pharmacist validation, and final delivery to the user.



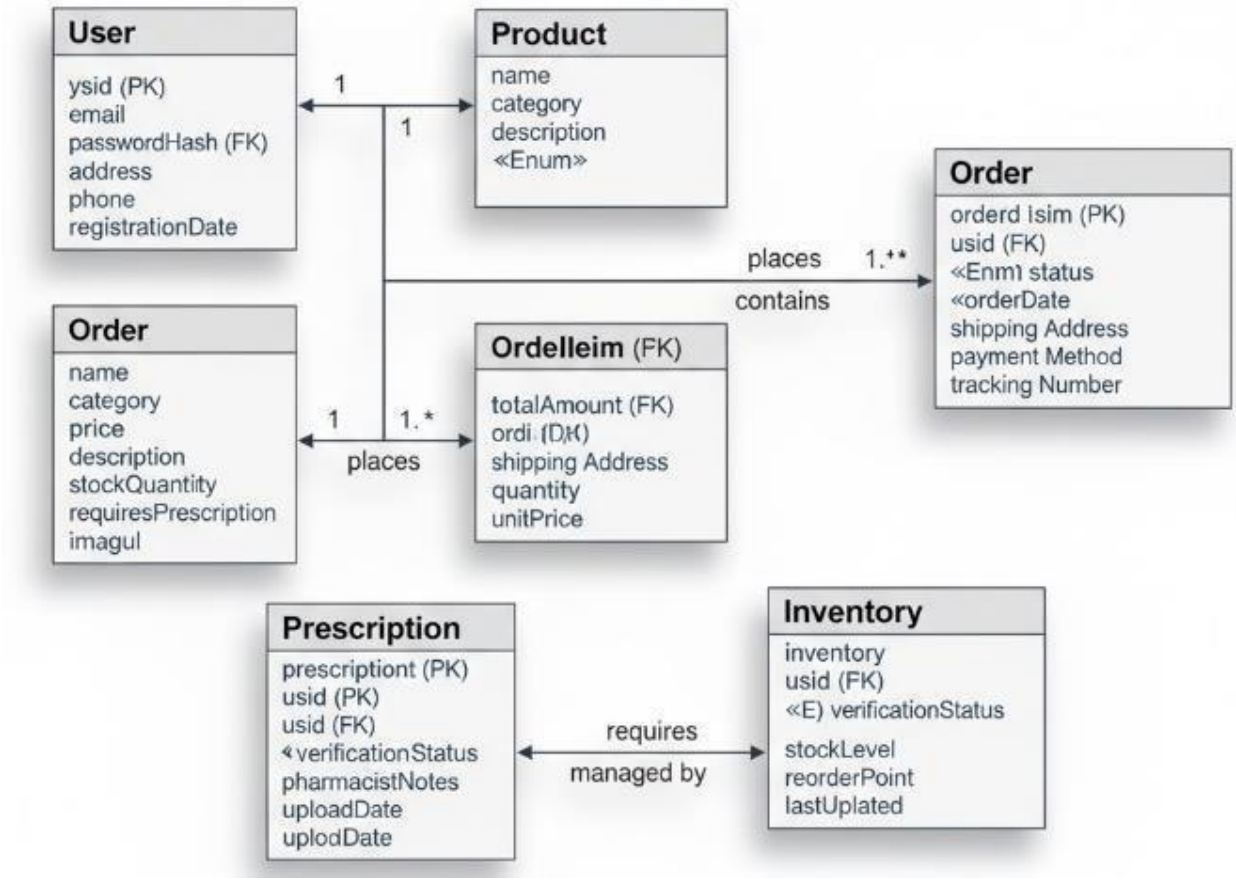
3.6.3 Sequence Diagram:

This sequence diagram illustrates the step-by-step interaction between the user, system components, and roles (pharmacist and admin) during the online medicine ordering process. It shows how data flows from medicine selection to prescription validation and final order delivery.



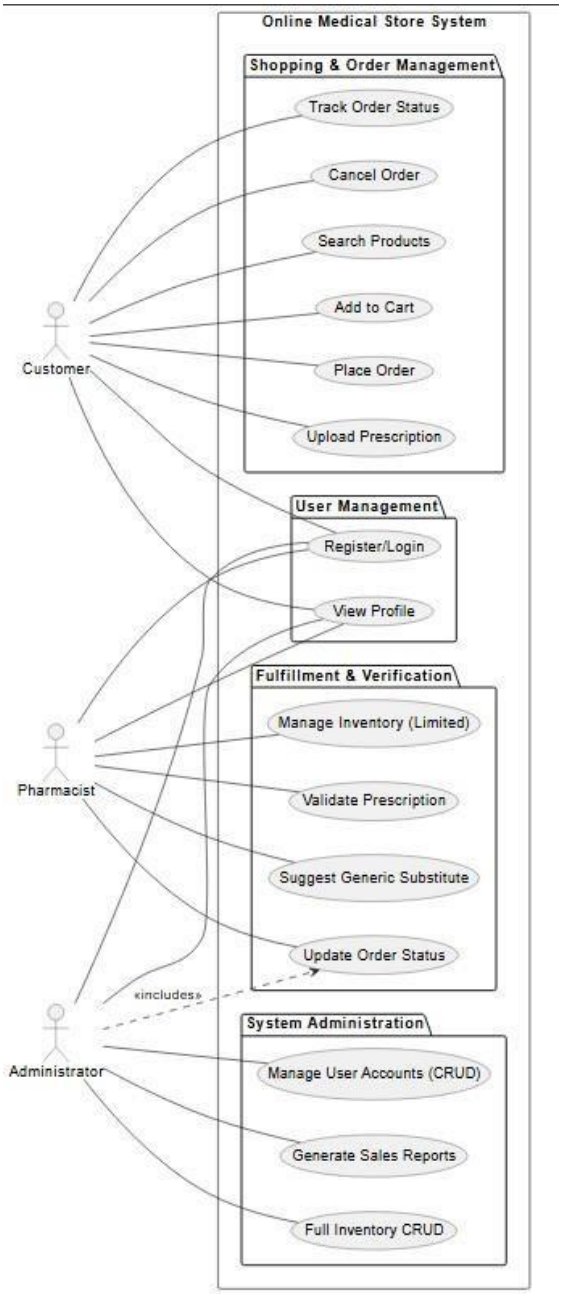
3.6.4 Class Diagram:

This UML Class Diagram serves as a **structural blueprint** for the Online Medical Store, illustrating how core data entities like **Users, Products, Orders, and Prescriptions** are defined and interconnected. It forms the essential foundation for the **MERN stack database design** and the application's overall logic.



3.6.5 Use Case Diagram:

This **UML Use Case Diagram** defines the **functional scope** of your **Online Medical Store** system. It clearly illustrates how primary users (Actors) like the **Customer, Pharmacist, and Administrator** interact with the system to perform essential activities such as **placing orders, validating prescriptions, and managing inventory**.

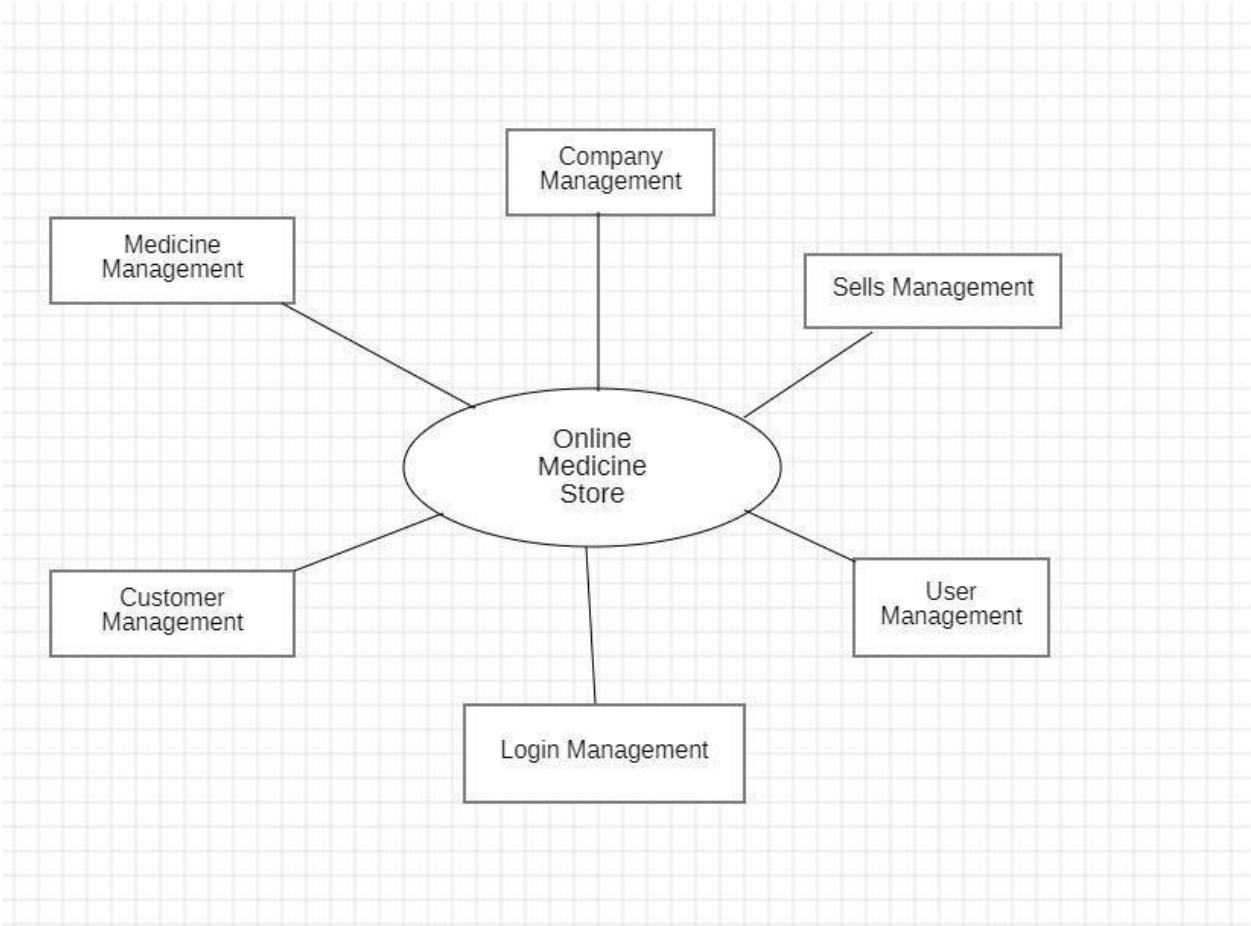


3.6.6 DATA FLOW DIAGRAM:

DFD LEVEL 0:

The Level 0 Data Flow Diagram (also known as the context diagram) provides a high-level overview of the entire online medical store system. It illustrates how external entities such as users, pharmacists, and

administrators interact with the system, and highlights the main data flows between these entities and the core system. This diagram sets the foundation for understanding the system's primary functions and boundaries without diving into internal processes.

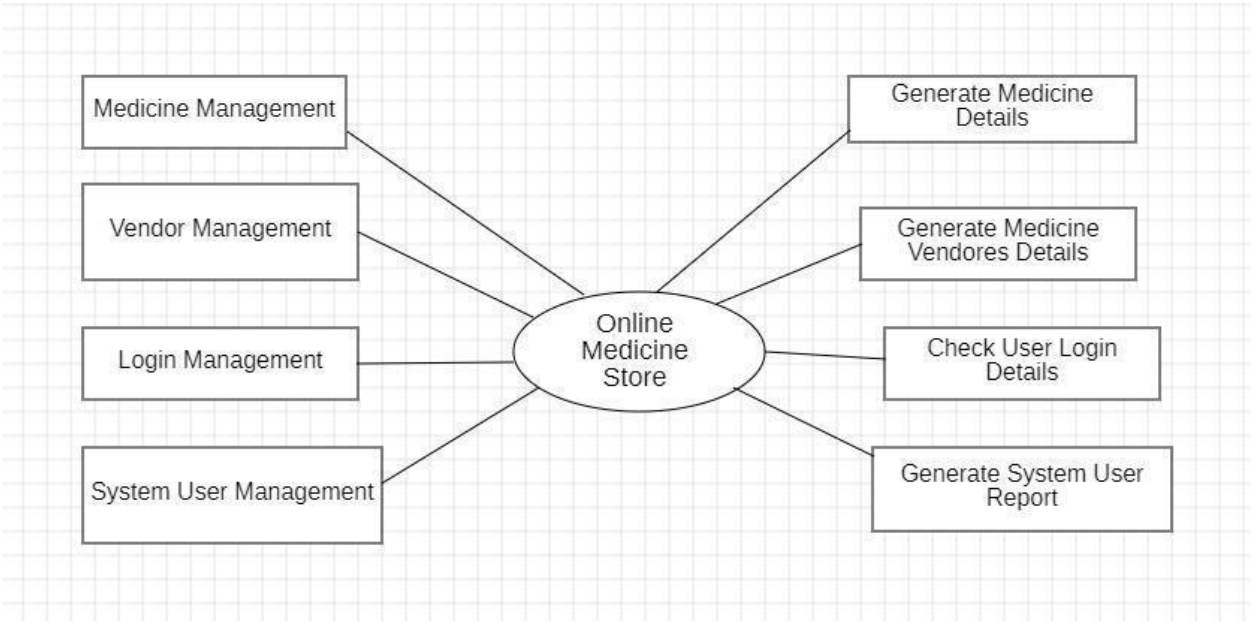


DFD LEVEL 0

DFD LEVEL 1:

The Level 1 Data Flow Diagram (DFD) expands upon the high-level processes outlined in the Level 0 DFD, breaking them down into more detailed sub-processes. It illustrates how user actions such as registration, medicine ordering, and prescription upload flow through various internal

components of the system. This level offers a clearer view of how data moves between users, the application backend, the database, and external services like payment gateways and logistics.

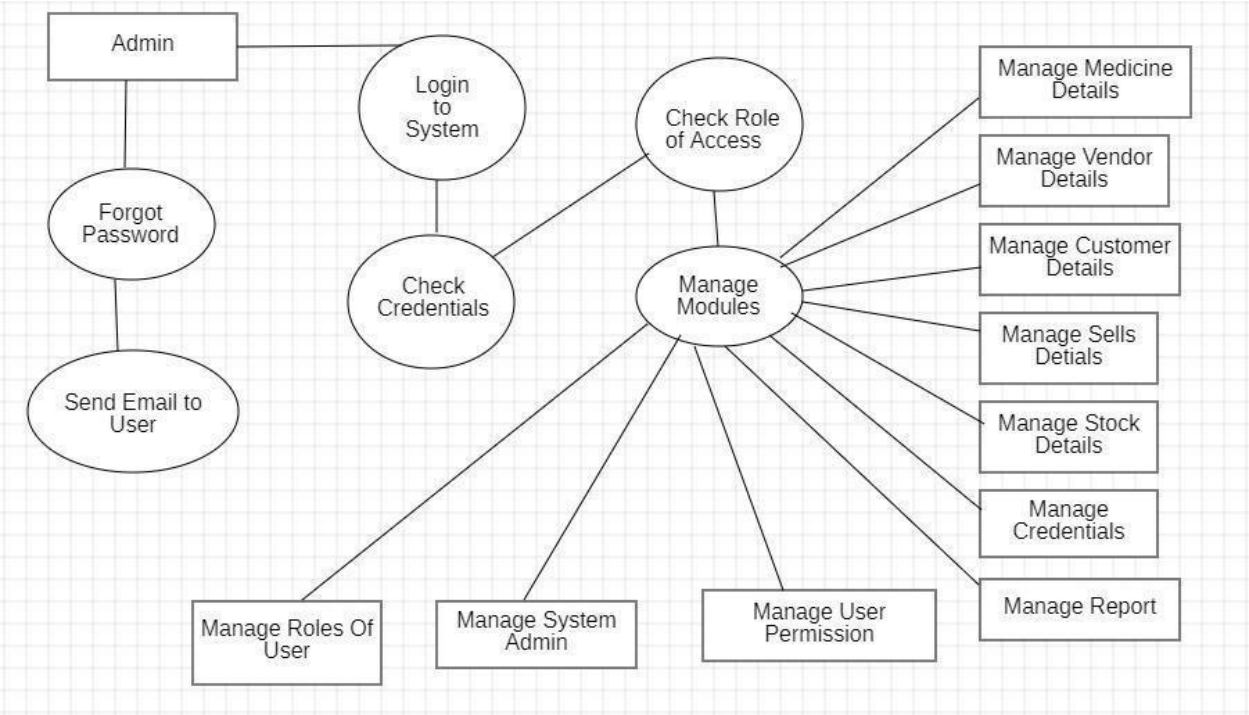


DFD LEVEL 1

DFD LEVEL 2:

DFD Level 2 (Data Flow Diagram Level 2) provides a deeper breakdown of the internal processes involved in specific modules, such as **user registration, medicine ordering, and prescription**

validation. It highlights detailed sub-processes like user authentication, medicine filtering, prescription upload validation by pharmacists, payment processing, and order confirmation. This level ensures granular visibility into how data flows between components like the frontend, backend services, and database in a more refined and modular manner.



DFD LEVEL 2

CHAPTER 4.SYSTEM DESIGN

The System Design chapter serves as the architectural blueprint for the pharmacy application, translating the requirements from Chapter 2 into concrete structural plans. It defines the major

components, how they interact, the data structures they rely on, and the underlying logic of the system. A well-executed system design ensures the application is robust, easy to manage, and capable of evolving over time.

4.1 Basic Modules:

The pharmacy application will be structured around a set of core modules, each dedicated to a specific functional area. This modular approach promotes loose coupling, making development and maintenance more manageable. The application fundamentally splits into three major user interfaces and several backend services.

1. User/Customer Module (Frontend)

This is the public-facing mobile application (iOS/Android) or web interface.

Authentication & Profile: Handles user registration, login, profile management, password reset, and securing user data.

Browsing & Search: Allows users to view categorized medicine lists, search by name, active ingredient, or condition, and view detailed product pages.

Cart & Checkout: Manages the shopping cart state (adding/removing items), applies discounts, calculates taxes/shipping, and initiates the checkout process.

Prescription Upload: Facilitates secure uploading of digital prescriptions (images/PDFs) and tracks their validation status.

Order Management: Allows users to view current and past order history, track live order status, and initiate returns/cancellations.

2. Admin Module (Web Panel)

A secure web application used by pharmacy staff and management.

Inventory Management: The central hub for adding new medicines, updating stock levels, managing product details, setting prices, and handling supplier information.

Order Processing: A dashboard for viewing all incoming orders, validating prescriptions, allocating stock, packing, and generating shipping labels.

User Management: Tools for managing customer accounts, handling support queries, and managing internal staff access permissions.

Reporting & Analytics: Generates reports on sales, inventory turnover, prescription validation rates, and customer behavior.

3. Pharmacist/Validator Module

A specialized interface, often integrated within the Admin Panel, for licensed personnel.

Prescription Review Queue: A prioritized list of uploaded prescriptions awaiting validation.

Validation Interface: Tools to view the uploaded prescription alongside the ordered items, mark the prescription as valid/invalid, and add notes or contact the customer if required.

4. Core Service Modules (Backend)

These services power the entire application.

Payment Gateway Service: Interfaces with third-party payment providers (e.g., Razorpay, Stripe) to securely process credit/debit cards, UPI, and net banking transactions.

Notification Service: Manages email, SMS, and push notifications for order status updates, prescription validation alerts, and promotional messages.

Logistics/Shipping Service: Coordinates with third-party delivery partners or manages in-house delivery schedules and tracking integration.

4.2 Data Design

The data design defines the structure, relationships, and integrity of the application's information. A Relational Database (SQL), such as PostgreSQL or MySQL, is typically chosen for its strong transactional integrity, crucial for e-commerce and inventory systems.

Key Database Entities (Tables)

Entity	Attributes	Relationships	Purpose
User	User_ID (PK), Name, Email, Phone, Hashed_Password, Address, Role (Customer/Admin/Pharmacist)	One-to-Many with Order	Stores all user credentials and profile information.
Product	Product_ID (PK), Name, Description, Category, Manufacturer, Price, Stock_Quantity, Prescription_Required (Boolean)	One-to-Many with Order_Item, One-to-One with Inventory	Central catalogue of all medicines and health products.
Order	Order_ID (PK), User_ID (FK), Order_Date, Total_Amount, Status (Pending/Validated/Shipped/Delivered), Payment_Method, Shipping_Address	One-to-Many with User, One-to-Many with Order_Item	Records transaction history and manages order lifecycle.
Order_Item	Item_ID (PK), Order_ID (FK), Product_ID (FK), Quantity, Unit_Price	Many-to-One with Order, Many-to-One with Product	Details the specific items and quantities within an order.
Prescription	Rx_ID (PK), User_ID (FK), Order_ID (FK), Image_URL, Validation_Status, Pharmacist_ID (FK), Validation_Date, Notes	One-to-One with Order	Manages the secure storage and tracking of prescription images.

Data Design Considerations

Normalization: Using 3NF (Third Normal Form) to eliminate data redundancy and ensure data integrity across transactions (e.g., ensuring a product's price is updated in only one place).

Indexing: Applying indices (especially on foreign keys and frequently searched attributes like Product Name and User ID) to optimize search and retrieval speeds.

Security: Hashing and salting passwords (never storing plaintext). Using encryption for sensitive data like payment details (though typically outsourced to the payment gateway).

4.3 Working Principle

The application follows a standard Client-Server architecture with a clear separation of concerns, employing a Microservices-style structure for the backend logic, though initially, it might be deployed as a single, well-organized Monolith for simplicity.

1. User Flow (Placing an Order with a Prescription)

1. Authentication: User logs in via the mobile app (Client).

2. Browsing: User searches for medicines and adds them to the Cart.

3. Prescription Upload: User selects the required prescription items and uploads a picture of the physical prescription. This image is stored securely on cloud storage (e.g., Amazon S3), and the URL is saved in the Prescription table. Checkout & Payment: User confirms the address and makes a payment via the Payment Gateway Service. The payment success triggers the creation of an Order record with a status of 'Pending Validation'.

4. Validation Queue: The new Order and associated Prescription are immediately placed in the Prescription Review Queue for the Pharmacist Module.

2. Pharmacist/Admin Flow (Order Fulfilment)

1. Validation: A Pharmacist logs into the Admin Panel, reviews the prescription image, verifies the order contents against it, and updates the Prescription status to 'Validated' or 'Rejected'.

2. Order Processing: If validated, the Admin Module updates the Order status to 'Processing'. Inventory is deducted (stock check/reservation).

3. Shipment: The order is packed, and the Logistics Service is engaged to generate a tracking ID. The Order status changes to 'Shipped'.

4. Notification: The Notification Service automatically sends an email/SMS/Push notification to the customer with the new status and tracking link.

4.4 Software Design

A Three-Tier Architecture is the standard for modern enterprise applications and is chosen here for clear separation of logic

.

1. **Presentation Tier (Frontend)**

Technology: React Native (for cross-platform mobile Website: iOS & Android) and React/Angular/Vue for the Admin/Web Panel.

Function: Responsible solely for the User Interface (UI), User Experience (UX), and translating user actions into API calls. It holds minimal business logic.

2. Application/Business Logic Tier (Backend)

Technology: A robust backend framework like Python/Django or Node.js/Express.js.

Function: Houses the core business logic (e.g., stock quantity checks, prescription validation rules, pricing logic, tax calculation, and order status transitions). This tier contains the RESTful APIs that the Presentation Tier calls.

3. **Data Tier**

Technology: PostgreSQL or MySQL (as the primary database) and a NoSQL store like Redis for caching frequently accessed data (e.g., popular product lists).

Function: Manages data storage, retrieval, and ensures data integrity. It is accessible only by the Application Tier.

Architectural Pattern: Model-View-Controller (MVC)

The MVC pattern will be applied within the Application Tier.

Model: Interfaces with the Data Tier (e.g., the Product Model manages all data interactions related to medicines).

View: Handled by the Presentation Tier (the user interface).
Controller: Handles incoming requests from the Presentation Tier, interacts with the Model to fetch/update data, applies business logic, and prepares the response.

4.5 Considerations for Design

Several key factors inform the design decisions to ensure the application meets nonfunctional requirements.

Security:

Authentication: Implement OAuth 2.0 or JWT (JSON Web Tokens) for stateless, secure session management.

Prescription Security: Use encrypted, secured cloud storage (e.g., private S3 buckets) for prescription images and implement strong access controls (only Pharmacists can view, users can only view their own).

Input Validation: Strict server-side validation against SQL injection and Cross-Site Scripting (XSS).

GDPR/HIPAA Compliance: Design data storage and handling processes with patient privacy and health data regulations in mind.

Scalability and Performance

Caching: Using Redis to cache product lists, popular searches, and session data to reduce database load.

Load Balancing: Deploying multiple instances of the Application Tier behind a load balancer to distribute traffic during peak sales.

Asynchronous Processing: Using background workers (e.g., Celery with Django) for nonimmediate tasks like sending notification emails, generating reports, or large inventory updates.

Usability (UX)

Mobile-First Design: Prioritizing the design for small screen sizes as the majority of users will use the mobile app.

Simple Checkout: Minimizing the steps required to complete a purchase (reducing cart abandonment).

Clear Status Updates: Providing real-time, unambiguous status updates for orders and prescription validation.

4.6 Future Enhancements

The system design must allow for future expansion without requiring a complete overhaul. Planned enhancements include:

AI/ML Recommendation Engine: Using machine learning to suggest personalized medication based on purchase history, browsing patterns, or prescription data (with explicit user consent).

Doctor Consultation Module: Integrating a video/chat feature allowing users to consult licensed doctors directly through the app before ordering, generating a valid digital prescription.

Subscription Service: A module for scheduled automatic refills of chronic medications, including recurring billing and automated stock checks.

CHAPTER NO 5

5.1 IMPLEMENTATION APPROCHES

An online medical store (e-pharmacy) can be implemented using different approaches depending on the business model, scale, and investment capacity. One common approach is the marketplace model, where the platform connects customers with multiple licensed pharmacies. In this model, the system acts as an aggregator, similar to PharmEasy or 1mg, and earns revenue through commissions while vendors manage their own inventory. Another approach is the inventory-based model, where the company purchases and stores medicines in its own warehouse and sells directly to customers, similar to Amazon Pharmacy. This model provides better quality control and higher margins but requires significant investment in storage, logistics, and regulatory compliance.

A hybrid model combines both strategies by stocking fast-moving medicines in a central warehouse while sourcing rare or location-specific drugs from partner pharmacies. Some platforms also implement a subscription-based approach for patients with chronic conditions, offering automatic monthly refills and reminders, as seen with Capsule. Additionally, integrating telemedicine services, like Practo, allows customers to consult doctors online and directly upload digital prescriptions for medicine orders.

From a technical perspective, the implementation requires a secure web or mobile application built with modern frontend and backend technologies, integrated payment gateways, prescription upload and verification modules, inventory management systems, order tracking, and compliance features such as drug license validation and data protection standards. Advanced features like AI-based prescription reading, drug interaction alerts, and automated reminders can enhance user experience and competitiveness. Overall, successful implementation depends on strong regulatory compliance, secure system architecture, efficient logistics integration, and a scalable technology framework.

5.2 CODING DETAILS AND CODE EFFICIENCY

4.HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
```

```
    <link rel="stylesheet" href="../styles/footer.css" />
  </head>
  <body>
    <hr />
    <div id="bot1">
      <div>
        <img
          src="https://assets.pharmeasy.in/web-
assets/dist/9ac2da37.svg?dim=0x72&dpr=1.25&q=100"
        />
        <h4>1 Lakh+ Products</h4>
        <p>
          We maintain strict quality controls on all our partner retailers, so
          that you always get standard quality products.
        </p>
      </div>
      <div>
        <img
          src="https://assets.pharmeasy.in/web-
assets/dist/2f258fe0.svg?dim=0x72&dpr=1.25&q=100"
        />
        <h4>Secure Payment</h4>
        <p>100% secure and trusted payment protection</p>
      </div>
      <div>
        <img
          src="https://assets.pharmeasy.in/web-
assets/dist/256fe591.svg?dim=0x72&dpr=1.25&q=100"
        />
```

```
    <h4>Easy Return</h4>
    <p>
      We have a new and dynamic return window policy for medicines and
      healthcare items. Refer FAQs section for more details.
    </p>
  </div>
</div>
<hr />
<div id="mob">
  <div class="mobimg">
    <img
      src="https://assets.pharmeasy.in/web-assets/dist/34a16ae8.jpg"
      alt="mobile App"
    />
  </div>
  <div class="app">
    <div id="txt">Download the App for Free</div>
    <div class="logflx">
      <div class="log">
        <img
          src="https://assets.pharmeasy.in/web-assets/dist/3380aedc.png"
          alt="Google Play"
        />
        <p>Google Play</p>
      </div>
      <div class="log">
        <img
          src="https://assets.pharmeasy.in/web-assets/dist/9bf5c576.png"
```

```
          alt="Apple Store"
        />
        <p>Apple Store</p>
      </div>
    </div>
  </div>
</div>

<div id="bot3">
  <div id="b3first">
    <div id="feat">
      <div>
        <h4>Company</h4>
        <p>Careers</p>
        <p>Blogs</p>
        <p>Partner with PharmEasy</p>
        <h4>Our Services</h4>
        <p>Order Medicine</p>
        <p>Healthcare Products</p>
        <p>Diagnostic Tests</p>
      </div>
      <div>
        <h4>Featured Categories</h4>
        <p>Covid Essentials</p>
        <p>Devices</p>
        <p>Nutriyion and Fitness Suppliments</p>
        <p>Personal Care</p>
        <p>Ayurvedic Care</p>
```

```
    <p>Baby and Mom care</p>
    <p>Skin Care</p>
    <p>Diabetic Care</p>
    <p>Sexual Wellness</p>
    <p>Short term Ailments</p>
    <p>Lifestyle Ailments</p>
    <p>Home Care</p>
    <p>Women Care</p>
    <p>Health Food and Drinks</p>
    <p>Ortho Care</p>
  </div>
  <div>
    <h4>Need Help</h4>
    <p>Browse All Medicines</p>
    <p>Browse All Molecules</p>
    <p>FAQs</p>
    <h4>Policy Info</h4>
    <p>Editorial Policy</p>
    <p>Privacy Policy</p>
    <p>Vulnerability Disclosure Policy</p>
    <p>Terms and condition</p>
    <p>Customer Support Policy</p>
    <p>Return Policy</p>
  </div>
</div>
<div id="social">
  <h4>FOLLOW US</h4>
  <img
```

```
      src="https://image.flaticon.com/icons/png/128/1384/1384031.png"
      alt="insta"
    /><img
      src="https://image.flaticon.com/icons/png/128/2111/2111392.png"
      alt="fb"
    /><img
      src="https://image.flaticon.com/icons/png/128/1384/1384028.png"
      alt="YT"
    /><img
      src="https://image.flaticon.com/icons/png/128/733/733635.png"
      alt="twit"
    />
    <iframe height="180" src="https://www.youtube.com/embed/a2HUAVYZ3Y8">
    </iframe>
  </div>
</div>
<div id="b3sec">
  <div id="pay">
    <h4>Our Payment Partners</h4>
    <span
      ><img
        src="https://assets.pharmeasy.in/web-assets/dist/af0c6812.svg"
        alt=""
    /></span>
    <span
      ><img
        src="https://assets.pharmeasy.in/web-assets/dist/0e010044.svg"
        alt=""
```

```
/></span>
<span
  ><img
    src="https://assets.pharmeasy.in/web-assets/dist/1066bb1f.svg"
    alt=""
/></span>
<span
  ><img
    src="https://assets.pharmeasy.in/web-assets/dist/fbea1701.svg"
    alt="" /></span
><span
  ><img
    src="https://assets.pharmeasy.in/web-assets/dist/3a1a533d.svg"
    alt="" /></span
><span
  ><img
    src="https://assets.pharmeasy.in/web-assets/dist/4733a49d.svg"
    alt="" /></span
><span
  ><img
    src="https://assets.pharmeasy.in/web-assets/dist/b278e536.svg"
    alt="" /></span
><span
  ><img
    src="https://assets.pharmeasy.in/web-assets/dist/0aca2077.svg"
    alt="" /></span
><span
  ><img
```

```
            src="https://assets.pharmeasy.in/web-assets/dist/44a51ca5.svg"
            alt="" /></span
          ><span
            ><img
            src="https://assets.pharmeasy.in/web-assets/dist/d058b00d.svg"
            alt="" /></span
          ><span
            ><img
            src="https://assets.pharmeasy.in/web-assets/dist/e9445364.svg"
            alt="" /></span
          ><span
            ><img
            src="https://assets.pharmeasy.in/web-assets/dist/6e09c713.svg"
            alt=""
          /></span>
        </div>
        <div id="copy">© 2021 PharmEasy. All Rights Reserved</div>
      </div>
    </div>
  </body>
</html>
```

LOGIN. HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>LOGIN</title>
    <link rel="stylesheet" href="../styles/login.css" />
```

```
  </head>

  <body>
    <header>
      <div class="head">
        <img
          src="https://assets.pharmeasy.in/web-assets/dist/fca22bc9.png"
          alt="logo"
        />
      </div>
      <div class="headright">
        <img
          src="https://assets.pharmeasy.in/web-assets/dist/1fe1322a.svg"
          alt="logo2"
        />
      </div>
    </header>
    <div class="login">
      <p>Quick Login /Register</p>
    </div>
    <div id="entry">
      <input type="text" placeholder="Enter your Phone number" id="num" />
      <button class="go" onclick="go()">Continue</button>
    </div>
    <div class="foot">
      By clicking continue, you agree with our <a href="#">Privacy Policy</a>
    </div>
  </body>
</html>
<script src="../src/login.js"></script>
<script src="../src/pincode.js"></script>
```

Signup.css

```
.plusimg {

  display: flex;

  background-color: rgb(133, 115, 225);
```

```
  width: 312px;
  margin-bottom: 30px;
  margin-top: 20px;
  border-radius: 5px;
}
.plusimg > img {
  width: 30px;
  height: 30px;
  padding-top: 20px;
  padding-left: 20px;
  cursor: pointer;
}
.plusimg > p {
  color: white;
  padding-left: 10px;
}
#left_box {
  height: 800px;
  border: 1px solid lightgrey;
  border-radius: 5px;
  width: 312px;
```

```
  margin-right: 40px;
  cursor: pointer;
}
.pay_main {
  display: flex;
  margin-left: 300px;
  margin-top: 60px;
}
.name1 {
  border-radius: 15px;
  background: teal;
  color: white;
  width: 30px;
  height: 30px;
  text-align: center;
}
.name1 > p {
  text-align: center;
  padding-top: 7px;
}
.bag {
```

```
  padding: 7px;
  display: flex;
  margin-top: 15px;
}

.bag > img {
  margin-left: 10px;
  height: 20px;
  width: 20px;
  cursor: pointer;
}
.bag_p {
  margin-top: -15px;
  margin-left: 20px;
}
.bag_p:nth-child(1) {
  color: lightgrey;
  padding-top: -20px;
}
.name {
  font-size: 20px;
```

```
  padding-top: 20px;
}

.profile > a {
  color: rgb(82, 76, 76);
  padding: 30px;
  font-weight: 600;
}

.edit_part {
  height: 550px;
  margin-top: 27px;
  border: 1px solid lightgrey;
  border-radius: 5px;
  width: 612px;
  margin-right: 40px;
  box-shadow: 0px 1px 15px lightgrey;
}
.entry {
  padding: 20px;
}
```

```
.entry > span {
  color: red;
}
.entry > input {
  width: 520px;
  height: 40px;
  margin-top: 15px;
}
.sub_for_offer {
  width: 540px;

  background-color: rgb(255, 247, 228);
  padding: 20px;
  height: 200px;
  margin-left: 17px;
}
.sub_for_offer > img {
  float: right;
  margin-top: -50px;
}
```

```
.sub_for_offer > input {
  width: 500px;
  height: 40px;
}
.sub_for_offer > button {
  height: 45px;
  width: 150px;
  margin-top: 15px;
  font-weight: 700;
  border-radius: 6px;
  background: rgb(255, 247, 228);
  border: 1px solid black;
}
#save {
  float: right;
  height: 38px;
  width: 114px;
  margin-top: 12px;
  margin-right: 16px;
  background: rgb(10, 87, 83);
  color: white;
```

```
  border: none;
  font-size: 17px;
  border-radius: 5px;
}

.card1 {
  border: 3px solid teal;
  border-radius: 10px;
  width: 37%;
  margin: 3%;
  padding: 3%;
}
.card1:hover {
  box-shadow: 0px 1px 15px teal;
}
#address1 {
  display: flex;
  margin: 8% 3%;
  box-shadow: 0px 1px 15px lightgrey;
  flex-wrap: wrap;
}
```

```
.card1 > h4,
.card1 > p,
.card1 > h5 {
  margin: 8px 0;
}
```

Signup.js

```
console.log("first_js");
let name = document.getElementById("username");
name.placeholder = "Enter Your name";


function sub() {
  let name = document.getElementById("name").value;
  let mobile = document.getElementById("mobile").value;
  let email = document.getElementById("email").value;


  let details = {
    name,
    mobile,
    email,
  };
```

```
let arr = localStorage.getItem("details");
if (name != "" && email != "" && mobile != "") {
  console.log("go");
  if (arr == null) {
    arr = [];
  } else {
    arr = JSON.parse(localStorage.getItem("details"));
  }
  console.log(arr);
  var k = 0;
  for (var i = 0; i < arr.length; i++) {
    var count = 0;


    for (key in details) {
      if (details[key] == arr[i][key]) {
        count++;
      } else {
        break;
      }
    }
    if (count == 3) {
```

```
        k++;
        break;
      }
    }
  }
  if (k > 0) {
    alert("Account already exists");
  } else {
    arr.push(details);
  }
  localStorage.setItem("details", JSON.stringify(arr));
}

function logout() {
  localStorage.removeItem("session");
  window.location.href = "index.html";
}

function savesess() {
  let name = document.getElementById("username").value;
  let arr = JSON.parse(localStorage.getItem("session"));
```

```
  console.log(name, arr[0]);
  arr[0].user = name;
  console.log(arr[0]);
  localStorage.setItem("session", JSON.stringify(arr));
}

function address() {
  let addr = JSON.parse(localStorage.getItem("addadd"));
  console.log(addr);
  let par = document.getElementById("address1");
  par.innerHTML = "";
  for (var i = 0; i < addr.length; i++) {
    let div = document.createElement("div");
    div.setAttribute("class", "card1");

    div.innerHTML = `
    <h3>Address ${i + 1}</h3>
<h4>Name : ${addr[i].name}</h4>
<h4>Number : ${addr[i].numa}</h4>
<p>Flat : ${addr[i].flat}</p>
<p>Street : ${addr[i].street}</p>
```

```
<h5>PINCODE : ${addr[i].pin}</h5>
    `;
    par.append(div);
  }
}
function saveadd() {
  console.log("add");
  let name = document.getElementsByName("Bill no")[0].value;
  let flat = document.getElementsByName("Flat number")[0].value;
  let street = document.getElementsByName("Street Number")[0].value;
  let pin = document.getElementsByName("Pincode")[0].value;
  let arr = JSON.parse(localStorage.getItem("session"));
  let numa = arr[0].num;
  console.log(name, flat, street, pin, numa);
  let addr = JSON.parse(localStorage.getItem("addadd"));
  if (addr == null) {
    addr = [];
  } else {
    addr = JSON.parse(localStorage.getItem("addadd"));
  }
  let adpro = {
```

```
    name,
    flat,
    street,
    pin,
    numa,
  };
  addr.push(adpro);
  localStorage.setItem("addadd", JSON.stringify(addr));
}

function showord() {
  console.log("Yeah");
  let par = document.getElementById("right_1");

  par.innerHTML = `
  <div class="right">
      <div><h3>MY ORDERS-HEALTHCARE</h3></div>
      <div class="cancel">
        <img
          src="https://assets.pharmeasy.in/web-assets/dist/025dc6fe.svg"
          alt="img"
```

```
    />

    <div>
      <p>USER</p>
      <span>Order Cancelled</span>
      <p>Dr.Odlin Digital Thermometer</p>

      <div class="reorder">
        <div>
          <p>Cancelled On</p>
          <p>23 june</p>
        </div>
        <div><p>Reorder</p></div>
      </div>
    </div>
  </div>
</div>
<div class="d_order">
  <select class="AllOrders">
    <option value="all">All Orders</option>
    <option value="Delivered Order">Deliverd Orders</option>
```

```
        </select>
      </div>
  `;
}
```

LOGIN.JS

```
/*function sign() {
  let form = document.getElementById("form");
  let name = form.name.value;
  let email = form.email.value;
  let num = form.num.value;
  let pass = form.pass.value;
  let check = form.check.checked;
  console.log(pass);
  if (name == "" || email == "" || num == "") {
    alert("One or more fields are empty");
    return;
  }
  if (pass.length < 8) {
    console.log(pass.length);
    alert("Password Must be atleast 8 Characters");
    return;
```

```
}
if (check == false) {
  alert("You've not agreed to our Terms and Conditions");
  return;
}

let user = {
  name,
  email,
  num,
  pass,
};
let arr = JSON.parse(localStorage.getItem("userarr"));
if (arr == null) {
  arr = [];
  arr.push(user);
} else {
  arr = JSON.parse(localStorage.getItem("userarr"));
  var k = 0;
  for (i = 0; i < arr.length; i++) {
    var c = 0;
```

```
    for (key in user) {
      if (arr[i][key] == user[key]) {
        c++;
        break;
      }
    }
    if (c > 0) {
      k++;
      break;
    }
  }
  if (k > 0) {
    alert("Account already exists");
    return;
  } else {
    arr.push(user);
    window.location.href = "index.html";
  }
}

localStorage.setItem("userarr", JSON.stringify(arr));
```

```
}

function log() {
  let arr = JSON.parse(localStorage.getItem("userarr"));
  let form = document.getElementById("logform");
  let nam = form.name.value;
  let pass = form.pass.value;
  console.log(nam, pass, arr);

  for (var i = 0; i < arr.length; i++) {
    if (arr[i].name == nam || arr[i].email == nam) {
      if (arr[i].pass == pass) {
        console.log("Loged In as " + arr[i].name);
        window.location.href = "index.html";
      }
    }
  }
}*/
//starts from here----->
function go() {
  //console.log("lets go");
```

```
let num = document.getElementById("num").value;
//console.log(num);
if (num.length < 10 || num.length > 10) {
  alert("Enter a valid Mobile Number");
  return;
}
let number = { num };
let arr = JSON.parse(localStorage.getItem("pharmuser"));
if (arr == null) {
  arr = [];
} else {
  arr = JSON.parse(localStorage.getItem("pharmuser"));
}


arr.push(number);
localStorage.setItem("pharmuser", JSON.stringify(arr));


let parent = document.getElementById("entry");
parent.innerHTML = "";
var a = ran();
var b = ran();
```

```
  var c = ran();
  var d = ran();
  alert(`Your OTP is ${a} ${b} ${c} ${d}`);
  console.log(a, b, c, d);
  parent.innerHTML = `
    <h4 style="padding-left:20px"> Enter OTP has been sent to ${num}</h4>
    <div class="otpflex">
    <input type="text" id="otp1" />
    <input type="text" id="otp2" />
    <input type="text" id="otp3" />
    <input type="text" id="otp4"/>
    </div>
    <div class="resendflex">
    <p onclick="numc()">Change Number</p>
    <p onclick="resend(${a},${b},${c},${d})">Resend OTP</p>
    </div>
    <button class="go" onclick="logged(${a},${b},${c},${d})">Continue</button>
    `;
}

function numc() {
```

```
  let parent = document.getElementById("entry");
  //console.log(par);
  //console.log("Return");
  parent.innerHTML = `<input type="text" placeholder="Enter your Phone number" id="num" />
        <button class="go" onclick="go()">Continue</button>
`;
}

function resend(a, b, c, d) {
  //console.log("Helloe");
  alert(`Your OTP is ${a} ${b} ${c} ${d}`);
}

function ran() {
  return Math.floor(Math.random() * 10);
}

var loog = false;
function logged(a, b, c, d) {
  let o1 = document.getElementById("otp1").value;
  let o2 = document.getElementById("otp2").value;
```

```
let o3 = document.getElementById("otp3").value;
let o4 = document.getElementById("otp4").value;

console.log(o1, o2, o3, o4);
if (o1 == a && o2 == b && o3 == c && o4 == d) {
  loog = true;

  document.querySelector(".popup").style.display = "none";
  //Never use this as it refreshes the whole effin page
  //window.location.href = "mainlandingpage.html";
  console.log(loog);
  let ruser = document.getElementById("useruser");
  ruser.style.display = "block";
  let logruser = document.getElementById("makemeuser");
  logruser.style.display = "none";

  let arr = JSON.parse(localStorage.getItem("pharmuser"));
  let session = JSON.parse(localStorage.getItem("session"));
  session = [];
  let us = arr[arr.length - 1];
  session.push(us);
```

```
    localStorage.setItem("session", JSON.stringify(session));
    console.log(session);
  } else {
    alert("Enter Valid Number and OTP sent");
  }
}

// function change() {
//   let loguser = document.getElementById("useme");
//   console.log(loguser);

//   loguser.innerText = "User";
// }
let session = JSON.parse(localStorage.getItem("session"));
let ruser = document.getElementById("useruser");
ruser.style.display = "none";
if (session.length > 0) {
  let ruser = document.getElementById("useruser");
  ruser.style.display = "block";

  let logruser = document.getElementById("makemeuser");
```

```
  logruser.style.display = "none";
}
console.log(session[0]);

if (session[0].user != null) {
  console.log(ruser);
  ruser.innerText = session[0].user;
}
```

Cart.js

```
var pri = Math.floor(Math.random() * 3000);
var offer = Math.round(Math.random() * 0.3 * 100) / 100;
var mrp = Math.floor(pri / (1 - offer));

console.log(mrp, offer, pri);

let prod = JSON.parse(localStorage.getItem("showproduct"));
console.log(prod);

let img = document.getElementById("slide");
let img1 = document.getElementById("img1");
```

```
let img2 = document.getElementById("img2");
let img3 = document.getElementById("img3");

let name = document.getElementById("tabname");
let company = document.getElementById("tabcompany");

let stock = document.getElementById("stock");

let actual = document.getElementById("actual");
let orig = document.getElementById("original");
let off = document.getElementById("taboff");

img.src = prod.imag;
img1.src = prod.img1;
img2.src = prod.img2;
img3.src = prod.img3;

name.textContent = prod.taga;
company.textContent = prod.company + " PVT LTD";

actual.textContent = `₹${prod.pri}*`;
```

```
orig.textContent = `₹${prod.mrp}`;
off.textContent = `${Math.round(prod.off * 100)}% OFF`;

if (prod.stock < 0.25) {
  stock.textContent = "Currently Unavailable, back in stock SOON";
  stock.style.color = "grey";
} else if (prod.stock < 0.5) {
  stock.textContent = `Only ${Math.ceil(
    prod.stock * 25
  )} Left In Stock. HURRY!`;
  stock.style.color = "red";
} else if (prod.stock > 0.75) {
  stock.textContent = "IN STOCK";
  stock.style.color = "green";
} else {
  stock.textContent = "Only Few Left ";
  stock.style.color = "blue";
}

function cartme() {
  console.log(prod);
```

```
let arr = JSON.parse(localStorage.getItem("pharmcart"));
if (arr == null) {
  arr = [];
  arr.push(prod);
} else {
  arr = JSON.parse(localStorage.getItem("pharmcart"));
  var k = 0;
  for (i = 0; i < arr.length; i++) {
    var c = 0;
    for (key in prod) {
      if (arr[i][key] == prod[key]) {
        c++;
      }
    }
    if (c > 6) {
      k++;
      break;
    }
  }
  if (k > 0) {
    alert("Medicine exists in your cart");
```

```
      return;
    } else {
      arr.push(prod);


      //   window.location.href = "index.html";
    }
  }
  alert("Medicine Successfully Added to your cart");
  document.getElementById("carted").innerText = "Added to Cart";
  localStorage.setItem("pharmcart", JSON.stringify(arr));
}


let arr = JSON.parse(localStorage.getItem("pharmcart"));
console.log(arr.length);
let lang = 0;
if (arr == null || arr.length == 0) {
  lang = 0;
} else {
  lang = arr.length;
}
let sh = document.getElementById("showcart");
```

```
sh.textContent = `${lang} Items in cart`;
```

CartPay.js

```
console.log("Oye");


let arr = JSON.parse(localStorage.getItem("pharmcart"));
let car = document.getElementById("carter");
var sum = 0;


if (arr == null) {
  arr = [];
  let car = document.getElementById("carter");
  car.innerHTML = `<div class="cart_box">
        <div class="cart_box1">
          <img
            src="https://assets.pharmeasy.in/web-assets/dist/8bfde66a.svg"
            alt=""
          />
          <p>Flat 15% off</p>
          <img
            src="https://assets.pharmeasy.in/web-assets/dist/adb9b646.svg"
            alt=""
```

```
    />
    <p>1 Lakh+Products</p>
    <img
      src="https://assets.pharmeasy.in/web-assets/dist/9279f530.svg"
      alt=""
    />
    <p>Easy Return</p>
  </div>
  <div class="cart_box2">
    <p>Search for Medicines/Healthcare Products</p>
  </div>
  <div class="cart_box3">
    <p>Select Pincode</p>
    <img
      src="https://image.flaticon.com/icons/png/128/25/25243.png"
      alt=""
    />
    <div></div>
    <input
      type="text"
      placeholder="e.g. Ecosprin, Telma or Colgate"
```

```
            class="Search"
          />
          <a class="search" href="#"
            ><img
              src="https://image.flaticon.com/icons/png/128/2089/2089805.png"
              alt=""
          /></a>
        </div>
      </div>
      <div class="cart_box_side">
        <h2>
          Your Cart is <br />
          Empty
        </h2>
        <img
          src="https://assets.pharmeasy.in/web-assets/dist/bb14593e.svg"
          alt=""
        />
      </div>`;
  } else {
    for (var i = 0; i < arr.length; i++) {
```

```
  console.log(sum, arr[i].pri, arr[i].qty);
  sum += arr[i].pri * Number(arr[i].qty);
}
let del = 0;
if (sum > 500) {
  del = 0;
} else {
  del = 50;
}
app(arr);
let but = document.getElementById("butcont");
but.innerHTML = `
  <input id="coup" type="text" ondblclick="appli()" placeholder="APPLY COUPON">
   <div class="coups">
      <p>Flat 20% Off</p>
      <p>AKSHAY20</p>
    </div>
    <div class="coups">
      <p>Flat 25% Off</p>
      <p>VANDNA25</p>
    </div>
```

```
  <div class="coups">
    <p>Flat 30% Off</p>
    <p>SWETHA30</p>
  </div>
  <div class="coups">
    <p>Flat 50% Off</p>
    <p>MASAI50</p>
  </div>
<button id="proceed" onclick="logValidate()">Proceed To Pay</button>
<div id="fri">Free Delivery for Order Above 500</div>
<h2>Order Summary</h2>
<div id="det">
  <div class="ord">
    <p>Cart value</p>
    <p id="cartvalue">₹${Math.round(sum)}</p>
  </div>
  <div class="ord">
    <p>Delivery Charge</p>
    <p id="del500">${del}</p>
  </div>
  <hr />
```

```
      <div class="ord">
        <h3>Amount to be paid</h3>
        <h3 id="totalcart">₹${Math.round(sum) + del}</h3>
      </div>
    </div>
    `;
}

function logValidate() {
  let x = localStorage.getItem("session");
  if (x == undefined) {
    alert("Please Log-In to Proceed");
  } else {
    window.location.href = "paynow.html";
  }
}

function app(arr) {
  let car = document.getElementById("cont");
  car.innerHTML = "";
  for (var x = 0; x < arr.length; x++) {
```

```
let div = document.createElement("div");


div.innerHTML = `
  <div class="parent">
<div class="fximg">
  <img
    src="${arr[x].imag}"
  />
</div>
<div class="fxtxt">
  <div class="fxtaga">
    <h2>${arr[x].taga}</h2>
    <button type="submit" onclick="dele(${x})">
      <img
        src="https://image.flaticon.com/icons/png/128/1214/1214428.png"
        alt="delete"
      />
    </button>
  </div>
  <div class="comp">
    <h4>${arr[x].company}</h4>
```

```
  <p class="del">
    Will be Delivered in ${Math.ceil(Math.random() * 7)} days
  </p>
</div>
<div class="both">
  <div class="lef">
    <select class="qty" id="sel${x}" onchange="selectme(${x})">
      <option value="1">QTY</option>
      <option value="1">1</option>
      <option value="2">2</option>
      <option value="3">3</option>
      <option value="4">4</option>
      <option value="5">5</option>
      <option value="6">6</option>
      <option value="7">7</option>
      <option value="8">8</option>
      <option value="9">9</option>
      <option value="10">Max 10</option>
    </select>
  </div>
  <div class="rig">
```

```
        <h5>${Math.round(arr[x].off * 100)}% OFF</h5>
        <span>MRP</span>
        <span class="mrp" id="mrp1${x}">${arr[x].mrp}</span>
        <span class="act" id="pri1${x}">₹${arr[x].pri}</span>
      </div>
    </div>
  </div>
</div>
    `;
  console.log(document.getElementById(`sel${x}`));
  div.setAttribute("class", "qtycard");
  div.setAttribute("id", `card${x}`);
  // let name = document.createElement("h3");
  // name.innerHTML = `${arr[x].taga}`;

  // let company = document.createElement("h5");
  // company.innerHTML = `${arr[x].company}`;
  // let url = document.createElement("img");
  // url.src = arr[x].imag;

  // let mrp = document.createElement("h3");
```

```
    // mrp.innerHTML = `${arr[x].mrp}`;
    // let off = document.createElement("h3");
    // off.innerHTML = `${arr[x].off}`;
    // let pri = document.createElement("h3");
    // pri.innerHTML = `${arr[x].pri}`;

    // div.append(url, company, mrp, off, pri);
    car.append(div);
  }
}

function dele(x) {
  console.log("Yes", x);
  console.log(sum);
  let arr = JSON.parse(localStorage.getItem("pharmcart"));
  console.log(arr[x]);

  //sum -= arr[x].pri * arr[x].qty;
  console.log(sum);
  if (sum > 500) {
    del = 0;
```

```
} else {
  del = 50;
}
let cartv = document.getElementById("cartvalue");
let totalcart = document.getElementById("totalcart");
let deli = document.getElementById("del500");
cartv.textContent = sum - arr[x].pri * arr[x].qty;
deli.textContent = del;
totalcart.textContent = sum - arr[x].pri * arr[x].qty;

arr.splice(x, 1);
console.log(arr.length);
let card = document.getElementById(`card${x}`);
card.remove();
localStorage.setItem("pharmcart", JSON.stringify(arr));
if (arr.length == 0) {
  localStorage.removeItem("pharmcart");
  // }
  // if (arr.length == null) {
  console.log("its comin");
  let car = document.getElementById("carter");
```

```
car.innerHTML = `<div class="cart_box">
    <div class="cart_box1">
      <img
        src="https://assets.pharmeasy.in/web-assets/dist/8bfde66a.svg"
        alt=""
      />
      <p>Flat 15% off</p>
      <img
        src="https://assets.pharmeasy.in/web-assets/dist/adb9b646.svg"
        alt=""
      />
      <p>1 Lakh+Products</p>
      <img
        src="https://assets.pharmeasy.in/web-assets/dist/9279f530.svg"
        alt=""
      />
      <p>Easy Return</p>
    </div>
    <div class="cart_box2">
      <p>Search for Medicines/Healthcare Products</p>
    </div>
```

```
    <div class="cart_box3">
      <p>Select Pincode</p>
      <img
        src="https://image.flaticon.com/icons/png/128/25/25243.png"
        alt=""
      />
      <div></div>
      <input
        type="text"
        placeholder="e.g. Ecosprin, Telma or Colgate"
        class="Search"
      />
      <a class="search" href="#"
        ><img
          src="https://image.flaticon.com/icons/png/128/2089/2089805.png"
          alt=""
      /></a>
    </div>
  </div>
  <div class="cart_box_side">
    <h2>
```

```
          Your Cart is <br />
          Empty
        </h2>
        <img
          src="https://assets.pharmeasy.in/web-assets/dist/bb14593e.svg"
          alt=""
        />
      </div>`;
  }
}

function selectme(x) {
  let arr = JSON.parse(localStorage.getItem("pharmcart"));
  let it = document.getElementById(`sel${x}`).value;
  console.log(it);
  let mrp = document.getElementById(`mrp1${x}`);
  let pri = document.getElementById(`pri1${x}`);
  mrp.textContent = `${arr[x].mrp * it}`;
  pri.textContent = `₹${arr[x].pri * it}`;
  arr[x].qty = it;
  console.log(arr[x]);
```

```
  sum = 0;
  for (var i = 0; i < arr.length; i++) {
    console.log(sum, arr[i].pri, Number(arr[i].qty));
    sum += arr[i].pri * Number(arr[i].qty);
  }
  document.getElementById("cartvalue").innerText = `₹${sum}`;
  document.getElementById("totalcart").innerText = `₹${sum}`;
  localStorage.setItem("pharmcart", JSON.stringify(arr));
}

// Apply Coupons Here
let coop = 0;
function appli() {
  //console.log("Hey ma");
  let coup = document.getElementById("coup").value;
  //console.log(coup);
  let div = document.createElement("div");

  let fin = document
    .getElementById("totalcart")
    .innerText.split("₹")
```

```
  .map(Number)[1];
console.log(fin);
if (coup == "AKSHAY20") {
  coop = fin * 0.8;
}
if (coup == "VANDNA25") {
  coop = fin * 0.75;
}
if (coup == "SWETHA30") {
  coop = fin * 0.7;
}
if (coup == "MASAI50") {
  coop = fin * 0.5;
}
div.innerHTML = `
<hr/>
<div class="ord">
      <h3>After applying Coupon</h3>
      <h3 id="totcoup">₹${Math.round(coop)}</h3>
    </div>
```

5.2.1 CODE EFFICIENCY

To improve the efficiency of online medical store webpage, you should focus on reducing redundancy, optimizing resources, and improving structure. First, avoid repeating CSS properties such as max-width: 1400px and margin: 0 auto across multiple sections; instead, create a reusable container class and apply it where needed. Move the large internal <style> block into a separate external CSS file to improve maintainability and browser caching. Since you are loading multiple CSS files, combine and minify them into a single optimized stylesheet to reduce HTTP requests and improve loading speed. Additionally, upgrade from Bootstrap 3 to Bootstrap 5, which removes the dependency on jQuery and reduces overall JavaScript size, improving performance and security. JavaScript files should be loaded using the defer attribute or placed before the closing </body> tag to prevent blocking page rendering. Images should be compressed, converted to modern formats like WebP, and loaded using the loading="lazy" attribute to enhance page speed, especially on mobile devices. Replace excessive ID-based CSS selectors with reusable class-based selectors to improve scalability and maintain cleaner styling. Also, use semantic HTML elements such as <header>, <nav>, <section>, and <footer> instead of excessive tags to improve SEO and accessibility. By applying these optimizations, your website will load faster, require fewer server requests, be easier to maintain, and provide a better user experience overall.

5.3 TESTING APPROACH

A proper testing approach for an online medical store should follow a structured strategy to ensure functionality, security, performance, and compliance. First, **unit testing** is performed to verify individual components such as login, medicine search, cart operations, prescription upload, and payment processing. After that, **integration testing** ensures that different modules like inventory management, payment gateway, order tracking, and notification systems work together correctly. Next, **system testing** validates the complete application workflow, from user registration to order delivery, under real-world scenarios.

Functional testing checks whether all features work according to requirements, including prescription validation, medicine filtering, and discount calculations. **UI/UX testing** ensures the website is responsive across devices and browsers. Since medical platforms handle sensitive data, **security testing** is critical; this includes testing for SQL injection, cross-site scripting (XSS), authentication flaws, and secure payment handling. **Performance testing** evaluates how the system behaves under high traffic, especially during peak sales or health emergencies, ensuring the server can handle concurrent users without crashing.

Additionally, **compliance testing** ensures the platform follows healthcare regulations, prescription verification rules, and data protection standards. **User Acceptance Testing (UAT)** is conducted with real users or stakeholders to confirm the system meets business expectations. Finally, **regression testing** is

performed after updates to ensure new changes do not break existing features. By combining these testing methods, the online medical store becomes reliable, secure, scalable, and ready for production deployment.

5.31 UNIT TESTING

Unit testing in an online medical store focuses on testing individual components or functions independently to ensure they work correctly before integrating them into the full system. Each module is tested in isolation, without relying on databases, APIs, or external services, often using mock objects or stubs. For example, the login module is tested to verify that valid credentials allow access and invalid credentials return proper error messages. The medicine search function is tested to confirm that it correctly filters products based on name, category, or dosage. The cart module is tested to ensure items are added, updated, and removed accurately, and that price calculations, discounts, and taxes are computed correctly.

Similarly, prescription upload functionality is tested to confirm that only valid file formats are accepted and file size limits are enforced. Payment calculation logic is tested separately to verify total amount computation before connecting to a real payment gateway. In inventory management, unit tests ensure stock deduction works properly when an order is placed. These tests are typically automated using frameworks such as JUnit for Java, pytest for Python, or Jest for JavaScript. By performing unit testing early in development, developers can detect bugs quickly, reduce debugging costs, and build a stable foundation for integration and system testing.

5.3.2 INTEGRATED TESTING

Integration testing in an online medical store verifies that different modules of the system work together correctly after unit testing is completed. Instead of testing individual components separately, this approach checks the interaction between connected modules such as the login system and user database, medicine search and inventory management, cart operations and stock updates, and order placement with the payment gateway. It ensures that data flows accurately between the frontend and backend APIs and that actions performed in one module correctly reflect in another. For example, when a customer places an order, integration testing confirms that the inventory is updated, the payment is processed, and confirmation notifications are sent successfully. It also validates the connection between prescription uploads and pharmacist review systems, as well as delivery tracking with logistics services. By identifying communication errors and interface issues between components, integration testing helps ensure a smooth and reliable workflow before proceeding to full system testing.

5.3.3 BETA TESTING

Beta testing in an online medical store is conducted after system testing and before the official launch of the platform. In this phase, the application is released to a limited group of real users, such as selected customers, pharmacists, or internal staff, to evaluate its performance in a real-world environment. The purpose of beta testing is to identify usability issues, unexpected bugs, performance problems, and user experience gaps that may not have been detected during earlier testing stages. Users interact with features like medicine search, prescription upload, cart management, payment processing, and order tracking, and provide feedback based on their experience. This testing also helps assess system stability under real usage conditions, including different devices, browsers, and network speeds. By collecting user feedback and monitoring system behavior, developers can fix remaining issues, improve functionality, and ensure the platform is reliable, user-friendly, and ready for full-scale deployment.

5.4 MODIFICATION AND IMPROVEMENTS

Modification and improvements in an online medical store involve analyzing feedback from testing phases, user interactions, and system performance to enhance functionality, usability, and overall efficiency. After beta testing, developers identify bugs, feature gaps, and performance bottlenecks that need correction. Modifications may include optimizing the search algorithm to provide more accurate medicine results, improving the prescription upload process for faster validation, and fixing errors in cart calculations or payment processing. Improvements can also focus on enhancing user experience, such as making the interface more intuitive, speeding up page load times, ensuring mobile responsiveness, and adding personalized recommendations based on user history. On the backend, system performance can be improved by optimizing database queries, implementing caching, and upgrading server infrastructure to handle more concurrent users. Security enhancements, including stronger encryption for sensitive data and stricter authentication mechanisms, are also critical. Continuous updates based on analytics, customer feedback, and evolving regulatory requirements ensure that the platform remains reliable, efficient, and user-friendly, ultimately increasing customer trust and satisfaction.

5.5 TEST CASES

1. User Authentication

- Verify registration with valid details.
- Verify login with correct credentials.
- Verify login fails with invalid credentials.
- Test password reset functionality.
- Verify logout works properly.

2. Medicine Search & Catalog

- Search medicine by name, brand, or generic name.
- Filter medicines by category, dosage, price, or availability.
- Verify alternative suggestions for out-of-stock medicines.
- Test sorting options (price low to high, high to low).
- Check medicine details page displays correct information.

3. Cart & Checkout

- Add medicine to cart.
- Update quantity of medicine in cart.
- Remove medicine from cart.
- Verify total price calculation including discounts and taxes.
- Proceed to checkout with valid payment methods.
- Validate error handling for insufficient stock.

4. Prescription Upload

- Upload valid prescription formats (PDF, JPG, PNG).
- Reject invalid file formats.
- Verify maximum file size validation.
- Check that uploaded prescriptions are correctly linked to the order.

5. Payment & Order Processing

- Test payment with multiple methods (credit/debit card, UPI, wallet).
- Verify successful order confirmation after payment.
- Check order cancellation and refund process.
- Validate email/SMS notification after order placement.

6. Inventory & Backend

- Verify stock deduction after order placement.

- Test low-stock alerts in the admin panel.
- Check inventory update after stock addition.
- Validate order status update flow (Pending → Dispatched → Delivered).

7. User Profile & Settings

- Update user profile details.
- Add or remove delivery addresses.
- Verify medical history or prescription history is correctly displayed.

8. Security & Compliance

- Test secure login and password encryption.
- Validate user session timeout.
- Check access restrictions for admin and pharmacist roles.
- Verify GDPR/HIPAA compliance for sensitive data.

9. Performance & Load

- Check system behavior under high concurrent users.
- Test page load times for catalog and search.
- Validate responsiveness on mobile devices.

CHAPTER 6 RESULT AND DISCUSSION

6.1 TEST REPORTS

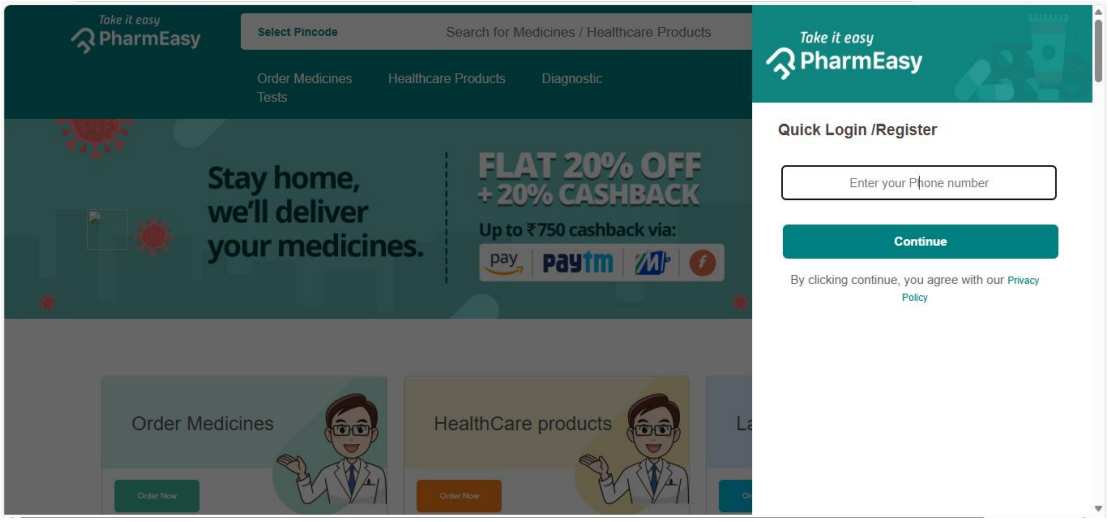
FIGURE 6.1

Test Case ID	Module	Test Description	Input / Action	Expected Result	Actual Result	Status
TC01	User Registration	Register new user	Fill registration form	Account created successfully	Account created successfully	Pass
TC02	User Login	Login with valid credentials	Enter username & password	Redirect to user dashboard	Redirected successfully	Pass
TC03	User Login	Login with invalid credentials	Enter wrong password	Error message displayed	Dashboard opened (unexpected)	Fail
TC04	Search Medicines	Search for medicine	Enter medicine name	List of matching medicines displayed	No results displayed	Fail
TC05	Add to Cart	Add medicine to cart	Click “Add to Cart”	Item added to cart	Item added successfully	Pass
TC06	Remove from Cart	Remove medicine from cart	Click “Remove” button	Item removed from cart	Item removed successfully	Pass
TC07	Upload Prescription	Upload prescription file	Select file & upload	File uploaded & visible to admin	File not uploaded	Fail
TC08	Online Payment	Pay for order	Enter payment details	Payment successful	Payment processed successfully	Pass
TC09	Order Tracking	Track order status	Enter order ID	Display current order status	Error message shown	Fail
TC10	Admin Add Medicine	Add new medicine to database	Enter medicine details	Medicine added successfully	Medicine added	Pass
TC11	Admin Delete Medicine	Delete existing medicine	Select medicine & delete	Medicine removed from database	Medicine deleted	Pass
TC12	Logout	Logout from system	Click “Logout”	Redirect to login page	Redirected successfully	Pass

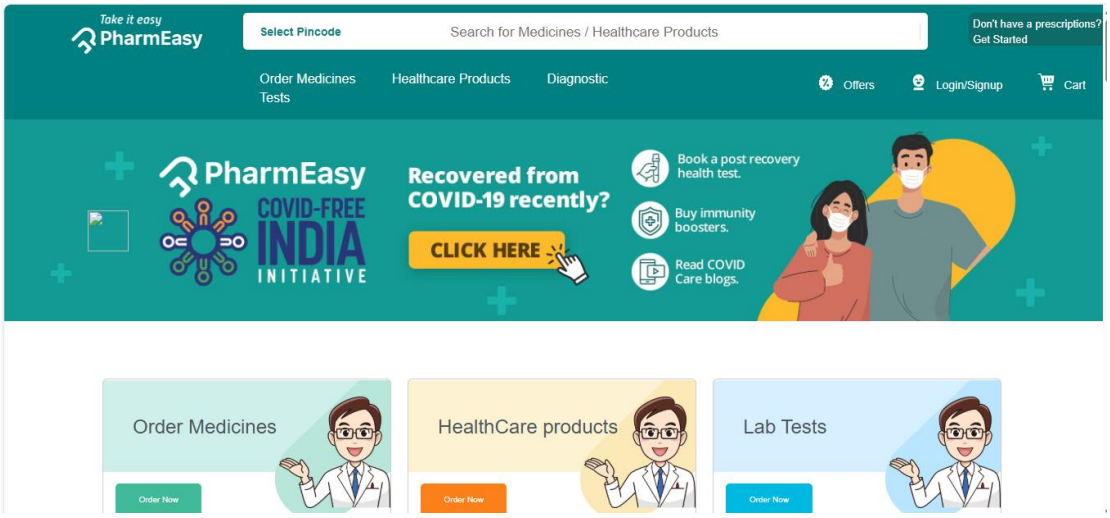
6.2 USER DOCUMENTAION

Online Medical Store is a web-based system that allows users to search, order, and pay for medicines online. Users can upload prescriptions, track orders, and manage their accounts, while administrators can add, update, or delete medicines and monitor orders. The system is user-friendly, secure, and designed to streamline the online medicine purchase process.

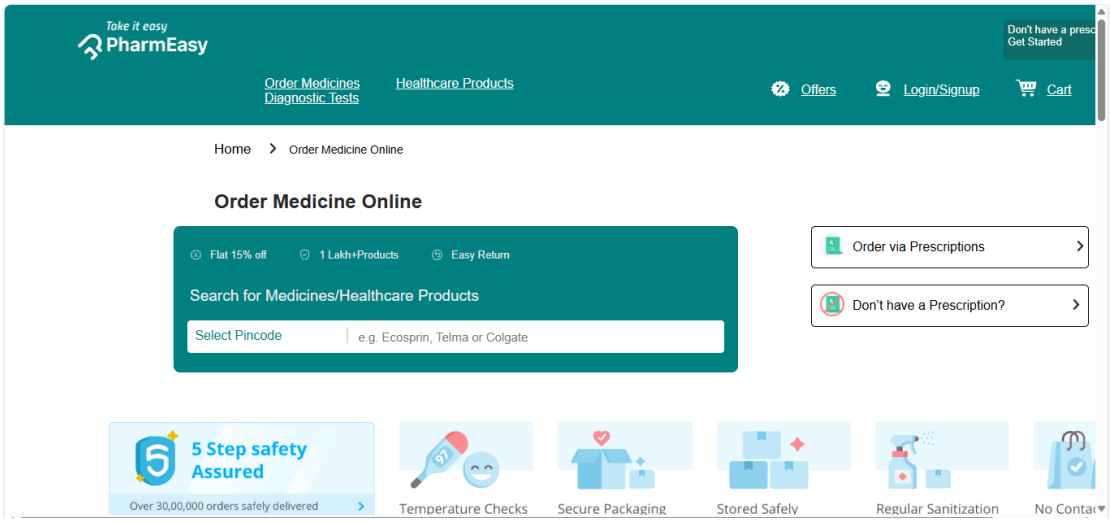
6.2.1 LOGIN PAGE



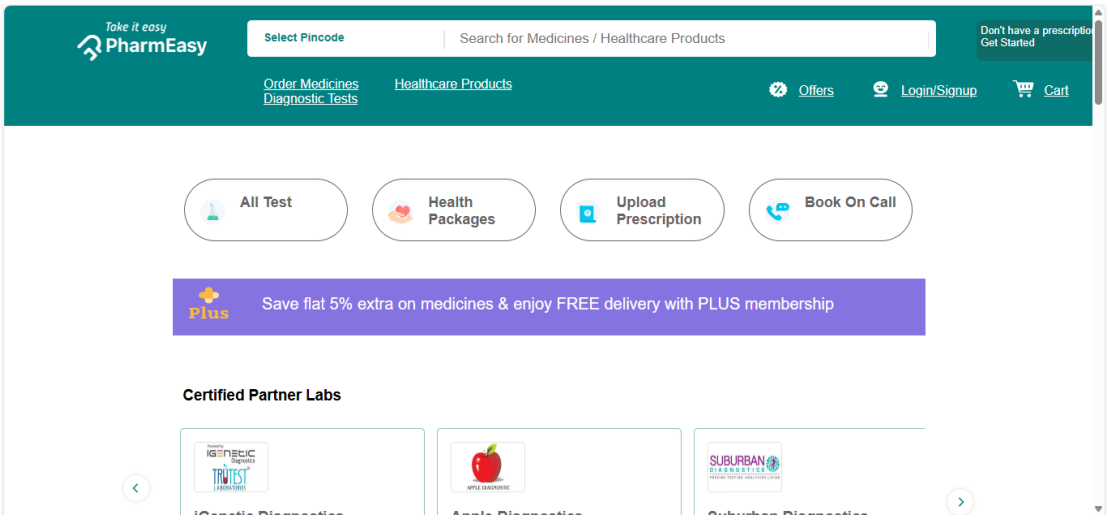
6.2.1 LOGIN PAGE



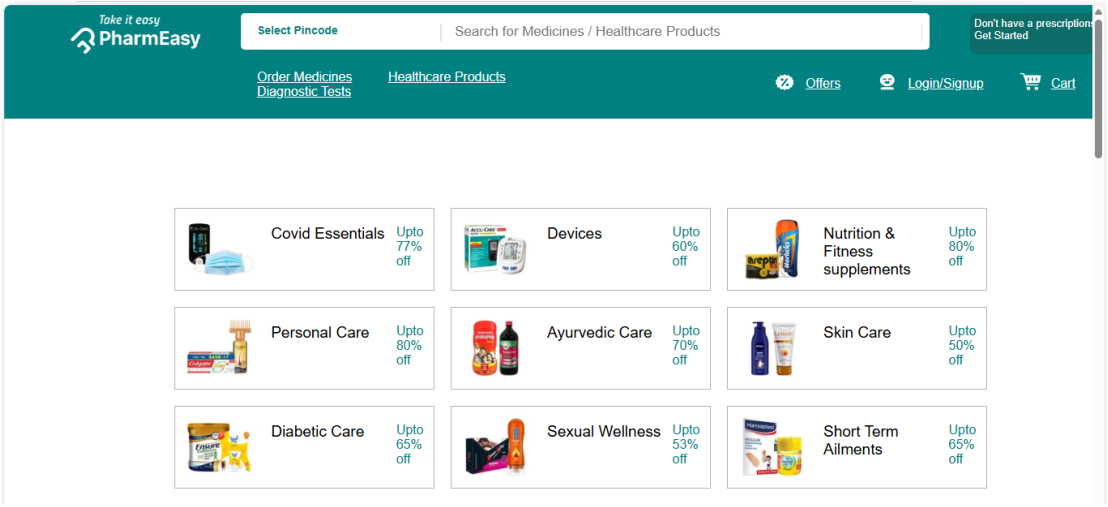
6.2.2 FRONT PAGE



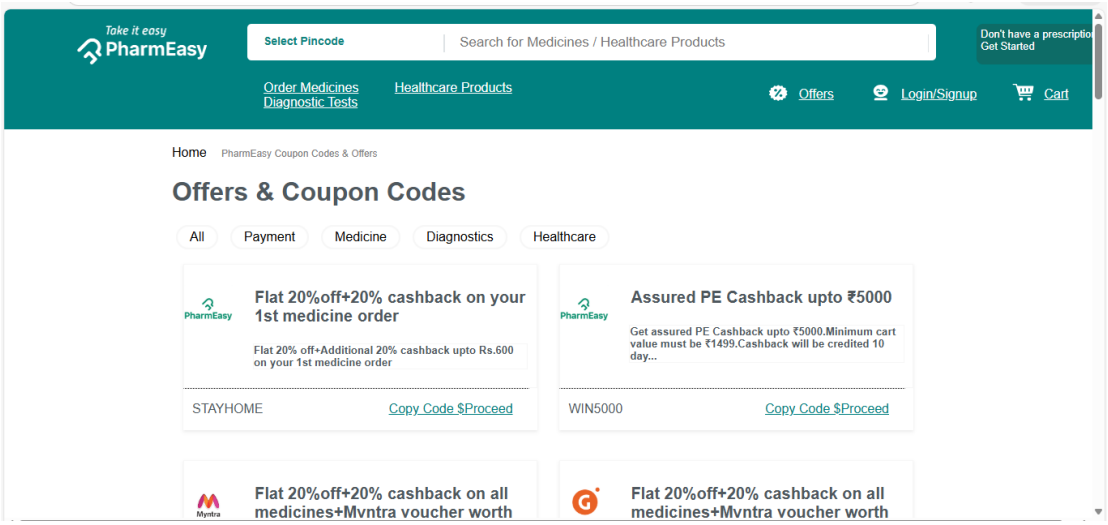
6.2.3 ORDER MEDICINE PAGE



6.2.4 ALL TESTS PAGES



6.2.5. PRODUCT CATEGERY



6.2.6 COUPON OFFER

CHAPTER 7 CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

The PharmEasy website was thoroughly evaluated and analyzed, focusing on user experience (UX), information architecture, and e-commerce efficiency. The study assessed key functionalities, including medicine search, prescription upload, and user dashboard interactions. The analysis confirmed that while PharmEasy provides a robust platform for online medicine and healthcare product delivery, critical areas require enhancement to improve overall usability and customer satisfaction. Specifically, improvements in the medicine search filter system, streamlining of the prescription upload process, and personalization of the user dashboard were identified as priorities. The findings indicate that addressing these areas can reduce user drop-off, increase conversion rates, and enhance repeat usage. Overall, the PharmEasy platform demonstrates strong functionality and market readiness, with clear opportunities for targeted improvements to solidify its position in the competitive digital healthcare landscape.

7.1.1 SIGNIFICANCE OF THE SYSTEM

The PharmEasy website plays a critical role in facilitating convenient, reliable, and accessible healthcare services in the digital space. By providing users with an online platform to search, order, and manage medicines and healthcare products, the system bridges the gap between traditional pharmacy services and modern e-commerce solutions. Its significance lies in enhancing healthcare accessibility, improving user convenience, and enabling efficient medication management for a diverse customer base. Furthermore, optimizing key functionalities—such as search, prescription upload, and personalized dashboards—directly contributes to increased user satisfaction, higher retention rates, and improved operational efficiency. Overall, the system not only supports individual health management but also strengthens PharmEasy's position as a leading digital healthcare platform in a competitive market.

7.2 LIMITATION OF THE SYSTEM

• The prescription upload process can be cumbersome, causing delays for users.
• The user dashboard lacks advanced personalization features, reducing engagement.
• Medicine search and filtering are not fully intuitive, affecting ease of use.
• The platform depends entirely on stable internet connectivity.
• Offline functionality, such as local order history, is not available.
• During peak traffic, response times may slow if backend performance is not optimized.
• Limited guidance for users on generic alternatives or substitutes.
• Notifications and reminders are basic and not highly customizable.
• Regulatory compliance requires frequent updates, which can temporarily affect service.

• Integration with third-party healthcare providers is limited.
• Accessibility features for differently-abled users are minimal.
• Overall, these limitations highlight areas for improvement to enhance usability, scalability, and customer satisfaction.

7.3 FUTURE SCOPE OF THE PROJECT

The immediate future work should focus on implementing and rigorously testing the recommended design enhancements to strengthen user experience, engagement, and platform efficiency. Key initiatives include:

• **Iterative UX/UI Redesign:** Develop high-fidelity prototypes to streamline the prescription upload process and enhance the medicine search and filtering system, ensuring intuitive navigation and reduced user friction.

• **Personalization Engine:** Explore and integrate an AI-driven personalization framework to deliver tailored product recommendations, targeted health content, and user-specific reminders based on historical behavior and preferences.

• **Mobile-First Optimization:** Conduct comprehensive testing and optimization to guarantee consistent, high-performance functionality across all mobile devices, reflecting the platform's predominantly mobile user base.

• **Long-Term Strategic Expansion:** Evaluate the feasibility of incorporating teleconsultation services directly within the PharmEasy ecosystem, creating a seamless, end-to-end digital healthcare platform that unifies medicine delivery, personalized care, and medical consultation.

These initiatives aim to not only improve immediate usability and engagement but also position PharmEasy for sustainable growth in the evolving digital healthcare landscape.

REFERENCE

1. **Business Model Profitability** — *PharmEasy revenue edge*
   🔗 https://stockify.net.in/blog/what-makes-pharmeasys-business-model-profitable-from-its-competitors/
2. **Search UX Case Study** — *PharmEasy search redesign*
   🔗 https://yashdafre.medium.com/ux-case-study-improving-the-pharmeasy-search-and-purchase-experience-e3e12a49e38d
3. **SWOT Insights** — *PharmEasy strategic analysis*
   🔗 https://iide.co/case-studies/swot-analysis-of-pharmeasy/
4. **Information Architecture** — *Website structure principles*
   🔗 https://www.userlytics.com/resources/blog/website-information-architecture/
5. **Online Pharmacy Awareness** — *Consumer behavior & barriers*
   🔗 https://pmc.ncbi.nlm.nih.gov/articles/PMC10415225/
6. **E-Prescription Processing** — *Clinical informatics study*
   🔗 https://academic.oup.com/jamia/article/19/3/353/2909138
7. **Digital Health Market** — *India healthtech growth*
   🔗 https://www.grandviewresearch.com/industry-analysis/india-digital-health-market-report/